

Raw Query Bank — Light Refine V2.1 (2026.05)

Note: This document preserves the original 口述 content but organizes it with clearer section headers, sub-headings, and consistent structure for each story. No content has been removed — only restructured for readability. **V2.1 Update:** 全部 13 个故事都已经从最新口述材料中详细整理 (包括 Stories 8, 9, 10, 11, 12, 13 的完整内容, 这些在 V1 / V2 是占位或简略状态)。Mixed Chinese/English wording is preserved as in the original.

Career Overview — Microsoft → Atlassian

Microsoft 时期 (2013 - 2025 年 1 月, 接近 12 年)

我是 2013 年毕业后加入微软的, 主要工作了接近 12 年。我从 IC 一直成长到 **Senior Principal Manager**。

职业发展主线:

- **前两年:** Search Relevance
- **接下来约 6 年多:** 从零开始搭建 Bing 的 **Question Answering System**, 特别是 Machine Learning 和 Deep Learning 的 Solution, 一直到 Pre-train 和 Large Language Model
- **2022 年之后:** 开始扩展 scope, 团队从 Machine Learning / Data Science Team 变成了 **Full Stack Team**; 负责 Bing 的 **Knowledge Answer** 和 **Knowledge Experience**, 包括 Bing 和 Edge 的 bridge
 - 当时做了 **Right Rail Pen** (Answer Side Project), 用户在浏览网页时可以在 right rail 提供各种 insights
 - 2023 年大模型出现后, 这个 Pen 升级成了 Edge 的 Copilot
- **2022 年之后:** 扩展 Knowledge Experience, 其中一个非常重要的项目叫做 **Knowledge Card** —— 基本上所有 Machine Learning 相关的东西都是我负责的; 它有非常大的流量, 大概占 Bing 流量的 60%, 有 60 多 Million
- **2023 年中开始:** 孵化并启动 **Generative SERP**, 我们团队负责整个 Generative SERP 的 Content Quality, 包括 Backend Flow 和 Experience Wise
 - 与美国团队合作, 但我的整个团队是 Full Stack, 可以 End-to-End 地贡献和推动
 - Science Problem、Data Workflow Pipeline 都是我们团队负责的, 只有 UX 是和美国团队合作
 - Generative SERP 后来升级成 **Copilot Search**, 与 Copilot 也在做 Deep Integration
 - **Copilot Search** 比 **Google AMMOD** 早上线一两个月

Atlassian 时期 (2025 年 2 月至今, 约一年半)

2025 年 2 月, 我从北京的 Microsoft 直接 relocate, 加入了美国 Atlassian 的 AI 核心团队。

- **Title:** Head of Machine Learning Engineering
- **Scope:** Knowledge AI

Knowledge AI 的三个 Big Charter:

Charter 1: Jira Search and Agent — 25 FTEs

Jira 这个 space 本身就是一个非常 **entity-centric** 的 area。Jira 里面有非常多种类的 entity:

- Jira Tickets
- Jira Project
- Jira Dashboard
-(每个都像一个 entity)

所以 Jira Search 不是 traditional 的 Document Search,而是 **Entity Search**。如何 enrich entity 的 context 和各种 metadata 对 ranking 非常重要。它有结构化搜索,也有无结构化搜索,而且里面的 entity 数据和属性也比较异构化 —— 这也是 Jira Search 相比于 Web Search 的一个难点。

Jira Agent: Search 到 Agent 是一个比较平稳的过渡。Search 是基础,帮助用户做 Knowledge Discovery;在此基础上,用户可能有更复杂的 information gathering 要求,或者除了 information gathering 还有各种 task action 需求。Agent 是一个更 advanced 的 framework,会 leverage 我们 Search 的 API 去做 grounding,同时也会更好地表达整个 search 的 flow。

Charter 2: Knowledge Fundamental — 15 FTEs

包含 Atlassian Cross Product 多个产品都在用的一些 service:

- **At Mention / People Recommendation** —— 当你 @ 某个人时,不管你在任何 service,它都是一个 entity ranking problem。根据你的 contexts、各种信息和 input,找到最可能的一个或多个 candidates。这个 traffic 基本上是 top tier 的 service,cross product 支持 various experience。
- **Entity Linking** —— 比如用户在 Search、Chat 或者写 Confluence Page 时,文字里都会提到一些 entity。这种情况下我们可以根据上下文做 entity resolution。
 - 在 Enterprise 场景下这并不容易,因为不能完全基于 popularity 做 ranking,更多是基于用户的 collaboration 关系,并根据搜索/Chat/文章上下文,做成 personalized ranking。
 - 这个 service 非常重要,是 cross 各种 product 都在使用的。

关于 **Entity Linking** 的 turnaround: 在我来之前,这块其实是一个 **lowlight**,之前的 Manager 做得不太好。我来之后仅仅用了 6 个月,就把它从 lowlight 变成了 **highlight**,并且找到了合适的人去做这个事情。

原来 team 里真正对这个问题有理解的人很少 —— 原来的 TL 不理解,Manager 也不太懂。后来这两个人都被我 manager out 了。我帮 team 找到了合适的 TL 和 teammates,这是 Entity Linking 能从 lowlight 变成 highlight 的非常重要的原因。**核心还是在于及时止损。**

Charter 3: Smart Answer in Search — 25 FTEs

这是 Atlassian 内部的一种 generative search 体验,有点类似于 enterprise 版本的 AI Mode。

接手前后的 MAU 增长: - 接手前: ~一两百万 MAU(虽然内部已经是 top AI feature,但 distribution 困难) - 过去一年: 从 100+ K 增长到 1.5 - 1.6 Million —— 一个非常 dramatic 的增长

两大重要功绩 —— Quality 和 Experience Uplift:

1. Experience Uplift

- 我 proposed 了 **Generative UI Layer** —— 这在整个 industry 都比较 advanced,不是简单的 freestyle experience
- 我们需要考虑产品之间 experience 的 coherence 和 alignment,以及与 Design Principle 的一致性
- 必须在可控性 (Controllability)、智能 (Intelligence)、新颖程度 (Novelty) 之间找到平衡

2. 跨领域的探索与应用

- 这块工作延续了我在 Microsoft 时期作为 pioneer 的初步探索。尽管当时 model 能力还处于早期,我们仍然在受限条件下,比 Google 更早 ship 了我们的 Copilot Search,并提供了一个更 visual appealing 的体验
- 在现在的场景下,我把在 Consumer Space 的 learning transfer 过来。同时,我花了大量时间与 cross-functional team 达成 alignment,最终将我个人的 proposal 变成了整个 org 的 direction

Career Transition Context

微软中国时期 (近 12 年): - 我在微软中国工作了接近 12 年,期间负责带一个 local team(其实也是 remote team) - 团队协作挑战: Headquarter 在 Redmond(西雅图),虽然中国团队规模很大,但远程协作中想 showcase 自己的 impact 非常不容易 - 沟通障碍: 由于各种各样的原因,尤其是信息不对称带来的问题比较多

赴美发展 (去年年初至今): - 通过 O-1 签证(类似人才引进的方式)直接来到美国并换了工作 - 主要原因是因为在原公司待的时间太长了,也希望有机会能去做一些变化

故事 1: Cross-Region/Remote Work — 远程团队领导力

背景

在中国,在微软的远程团队工作了 11-12 年。

远程职位的被动性

在微软,这种远程职位其实会有很多被动的地方: - 很多事情的信息会有时差 - 很多重要的事情,如果你想承担更重要的职责,意味着只有你拥有独特的价值,这件事可能只能由你来做,他们才会把主导权和驱动角色交给你 - 或者说,当美国团队忙不过来时,他们才会把任务交给中国团队

但在我从做 QA 项目开始,我们团队就扮演了非常主导的角色,特别是在机器学习和深度学习的应用、核心质量等方面。在与整个合作团队的合作中,我们处于领先地位。这意味着你的团队必须足够全面 —— 当别人做到 100 分时,你应该做到 120 分、150 分,甚至 200 分,这样才有可能获得同等的机会去领导一些有影响力的工作。

作为经理的核心 takeaways

1. 与美国团队的合作关系

- 需要与美国的团队成员保持良好合作关系,至少要有基本的信任
- 如果连基本的本职工作和合作关系都做不好,他们可能也不愿意支持团队的成长
- 因为我们团队的资源都是由美国合作团队资助的。如果做得不够好,他们可能会减少资助,或者不再继续增加资助
- 我这个团队是从零到一建立起来的,从最初的 5 个人,一直发展到 40+ 人的规模。每一次增长都代表着团队在某些方向上又跨出了一个新的高度

2. 信息差

- 如果存在信息差,作为领导者,你需要与美国那边的关键赞助人建立联系,让他们觉得你非常值得信赖

- 这样在很多情况下,他们会更早地把信息传达给你。否则,你可能会持续处于信息差中,错过一些机会

3. 追求卓越

- 如果你不想平庸,就应该追求 150% 甚至 120% 的效果
- 即使你扮演驱动角色,但由于你不在总部而是在远程团队,某些会议中可能会因时差无法参与。那么你的核心想法如何及时传达出来,而不是被别人绕过?
- 这需要建立非常强大的合作关系,让 PM、TPM、直接合作的工程师都愿意与你达成同盟
- **你必须找到几个盟友来支持你**,无论是单一的项目、想法,还是整个项目。否则如果你作为 PM 不在这些重要的讨论中,你的可见度等各方面都会受到比较大的影响

远程团队的特殊性 —— 为什么需要技术过硬 + 关系搞好

这个过程中也让我学到了很多 —— **不能只做眼前的事情,需要有更超前的思维去做孵化**。这也使得我们团队在之后的发展中有更强的潜力。甚至我们有些孵化项目在业界、在研究社区都能成为某种标杆,做成某种引领,大家可以学习、参考。

总结的核心要点

1. 建立信任 (Build Trust)

- 首先还是要建立信任,这样你才能得到盟友的赞助
- 在各个环节都可能得到赞助 —— 资源增长、承担责任、确保持续可见度和影响力

2. 人才追求 (Talent Density)

- 对人才的追求,对团队人才密度和能力的要求要非常高
- 我们团队当时人才密度非常高
- 现在虽然我离开了微软,我的同事们去了很多外部公司,在这些公司里都扮演了非常关键的角色
- 很多团队成员从最早做研究实习生开始,后来慢慢转正,他们的表现都非常出色

3. 孵化能力 — Compress and Extend

- 怎么才能做好孵化?合作方可能对你有很多要求,你也会很忙
- 一个很重要的思想是 **“压缩与扩展” (compress and extend)**:
 - 在一定时间内,想办法压缩一些事情,或者通过提升团队效率更快地完成
 - 团队成员能力足够强,他们也有意愿去承担风险,做一些孵化的事情
- 作为领导者,**找到一些好的、有前景的方向也非常重要** —— 你需要预测未来一个季度、半年甚至一年的发展,紧跟领先的趋势

4. 最大化影响力 (Maximize Impact)

- 怎么能够最大化自己的影响力,让团队有一个良性循环?
- 虽然我们有很多被动的地方,但我们要把这些被动转化为某种动力,反而让团队拥有更强的护城河或优势
- 这就是每个人单点的能力 + 团队协作的能力 + 整体的交付效率
- 我们当时与合作方合作时,**但凡遇到特别着急的事情、特别难啃的骨头,基本都会交给我们的团队去做**
- 这是长期积累和磨合的结果,而不是一蹴而就。我们积累了优秀的人才本身就是一个很好的资源,还有我们做事情的方式方法和工作流程

5. Think Big, Start from Small to Showcase Tangible Results

- 你可以 aim high, 有 big picture, 但执行时不能说我需要花 10 个 resource、10 个人去铺上去
- 你需要想办法用最简单的方法、最少的 resource 去 showcase 你的成果, 从而说服自己、说服团队
- 如果以后需要更多投入, 也需要让 partner team 知道我们可能需要调动一些 resource 去做别的事情

6. 眼光长远 — Scalable Solutions over Low-Hanging Fruits

- 在项目目标和想法上, 要尽可能追求可扩展 (scalable) 且能服务于中长期的方案, 而不只是关注容易达成的短期目标
- 这并不是说短期目标不做, 而是要时刻保持远见, 在繁琐的项目中理顺思路, 蹚出一条路来

7. Long-term, Long-Lasting Partnership

- 建立长期且持久的合作伙伴关系也非常重要
 - 我们在过去合作中建立的关系往往不是单点的 partnership, 未来它会辐射到非常多的项目
 - 比如我们跟一些团队在模型 (model) 和基础设施 (infrastructure) 上的合作, 最终会形成一个正循环
 - 大家不仅仅是谁是谁的 partner, 或者谁去贡献, 更是一种共同拥有 (co-own) 和共同驱动 (drive) 的关系
 - 一旦把事情做好, 大家都能获得很好的认可 (credit)
-

故事 2: QnA System — Overview (2016 - 2022)

项目概述

先说说我在微软很早开始的、非常重要的项目 —— 问答系统 (Question Answering System)。

- **时间跨度:** 2016 年 - 2022 年 (然后因为我们做到了 global 规模处于很好状态, 我们稍微缓了缓, 把精力挪去做 Knowledge Data & Experience)
- **2023 年大模型出现后:** 我们又反过头来 uplift 我们的问答系统
- **基本上,** 问答系统这件事贯穿了我整个微软生涯

完整的 Dual Lifecycle 经历

我经历了它完整的双生命周期:

1. **技术周期:** 模型技术的演进与迭代
2. **产品周期:** 从产品层面、用户感受、语言能力维度的产品升级周期

这个过程让我在 AI 领域获得了 “从做中学” (Learning by doing) 的快速成长, 而不仅仅是纯粹的科研或想法。也帮我更深切地理解了产品与科研之间的联系。

QnA 系统的发展阶段

1. **第一阶段: Rule-based System** - 最初是基于规则的系统, 后来演进到早期机器学习框架 - 那是一个基于树模型 (Tree-based) 的架构, 拥有 400+ 个特征 (features) 的排序器 (ranker)
2. **第二阶段: Deep Learning as Feature** - 比较早期的深度学习应用 - 我们将 DNN、LSTM 等当时主流模型作为特征引入, 并与 Partner 团队一起构建了离线框架

3. 第三阶段: Deep Learning Centric - 整个 pipeline 由深度学习模型搭建 - 没有很多传统的特征工程或基于树的模型 - 基本上是一个端到端 (end-to-end)、以深度学习为中心的 pipeline - 第一代 pre-trained model 时代 - 一、二、三阶段主要处理的还是英文

4. 第四阶段: 跨语言阶段 - 早期阶段先是在少数 top 语言上扩展 - 随后探索出 **universal solution**, 可以直接横扫 top 100 的语言范围

Organization 角度的复杂度

QnA 是一个为数非常罕见的、有三个 org 一起做的 feature, 这在 Microsoft 非常少见。

各 team 的侧重:

- **Team 1:** 负责 purely online QnA(query 必须实时生成)
- **Team 2:** 也负责 online, 但比较侧重 text-based(test-heavy)的 QnA
 - 对所有 query 基本普适
 - 提供兜底 solution(很多 query 上做得很好, 不仅仅是兜底)
- **Team 3:** 负责 **offline QnA**(query 提前收集, 答案 offline 生成后放到文档里)
 - 我之前的 team 主要就是以这个 offline QnA 为主, 跟我们美国那边一起

Offline Pipeline (2021 左右) 的问题

刚开始主要都是 batch processing pipeline, 当时面临很多问题:

- **当 query volume 巨大时:** 每次 batch refresh 都会产生极高的 GPU usage 峰值
- **核心问题在于数据的新鲜度 (freshness):** 无论如何设计 batch plan, 对 freshness 要求极高的 query 依然无解

这也是为什么我后来提出了 **Near Real-Time Pipeline** —— 这是 QnA 体系中非常重要的一个 story。

两种 pipeline 各自的局限:

1. **传统 Online Pipeline** —— 在消耗大量资源的情况下, 总是处于 quality suboptimal 的状态
2. **Offline Batch Pipeline:**
 - Refreshness 是严重短板
 - 包含 batch job management 在内的维护成本极高
 - 虽然尝试自动化, 但涉及多个 team 和 platform 的变更, pipeline 经常会挂掉, 很多时候仍需人工介入
 - 跨区域 (location) 的 communication 导致从 release 到 shipment 的整个周期和人力 cost 都居高不下

多语言扩展中的 Bottleneck

当英语做到不错的程度, 开始向多语言扩展时, batch inference 的 efficiency 就变成了 bottleneck。

- Global traffic 大致是其他所有语言加在一起才和英语持平
- 这意味着资源投入和 maintenance cost 感觉都要 double
- 但我们不可能继续 scale 更多的人去做这件事

在这种 multi-language 和 cross-lingual 场景下,重要问题主要分为两块: 1. **Model 角度** 2. 整个 **Pipeline 的 Efficiency 和 Effectiveness 角度**

Remote Team 的难点

作为 **AI leading 的产品**: - 从技术角度非常 exciting,很多机会 - 我们有机会在最早期 QnA 系统(还没进入机器学习阶段时)就加入,基本上从零开始参与 - 这样的经历本身有非常大的价值 —— 能做像 Bing 这么大的 consumer-scale 问答系统的 stack,在业界都是比较少有机会去做的 - 当时主要竞争对手是 Google,后来在国内,因为它需要非常强的搜索系统和 index 支持

作为 **remote team 的挑战** —— **Multi-team Competition**: - 美国这边已经有另外三个算法 team 也在参与 - 信息本身就是滞后的 - 高速发展,合作过程中势必会有 overlap,甚至有直接竞争 - 这种情况下,team 的 **velocity** 还有方向的前瞻性非常重要

早期阶段 (Pre-trained Model 之前 / Bert 时代): - 中国团队处于快速学习和成长的状态 - 同时要 **build trust** —— 这非常重要,为我们后来承担更大的 responsibility 做准备 - 否则即使效果好、方法好,partner 也未必会 support 我们去直接 lead

与 Google 的激烈竞争: - QnA 是 Bing 和 Google 三大赛 (Whole Page SBS) 的 critical feature - 当 Google 有一个很好的 pioneering feature,而 Bing 没有(或效果不那么好)时,immediately 三大赛就 lose - 不管你的其他部分多么好,impact 能 contribute 到最后 well-thought-out decision 里面的就已经非常少 - 如果想在这里面有更高的 voice、更强的 visibility,其实是比较难的事情

Joint Triage Meeting 的关键作用

我们当时 setup 了一个非常重要的 rule:

每周最多不超过两周,我们一定要参加 QnA 的 Joint Triage Meeting

Joint Triage 是各个 team 之间协调、make this shift work 的 community。所以整个 team 是 trained well 并且 high velocity 的。

人才策略

我当时找到了一些 talent —— 不仅 level 很高,有一些是 intern,但 quality 都非常高: - 在和高校尝试 build 的人脉、研究院/研究所的人脉 - 这种情况下我们就有比较好的人才输出

著名 intern 案例:

国内现在有几家做大模型的公司或 research lab,从我们团队出去的同事都 play 了 critical position。比如: - **DeepSeek** —— 它非常重要的部分是 code 的 training。这个 intern 就是我们联合培养、联合孵化的一个 intern - **CodeBERT** 那篇文章也是在那个阶段在做 pre-training,去 incubate - 现在有 **3000+** 引用 - 全球的很多研究者和开发者把这个工作作为一个基础 layer(基本起点) - 这已经不仅限于 influence Microsoft 的某个团队或某个产品,而是面向整个 industry,我们都有 leading impact

QnA 在 Bing Traffic 上的占比

英文市场: - 加入时: Coverage 基本可以认为是从零开始 - 后期: 涨到 **40% 左右** - Trigger Rate 和 Quality 高度相关 —— 我们要平衡 Coverage 和 Precision - Bing 上 QnA 的 Upper Bound (理论上限), 在英文环境下大概接近 **50%** - 高峰期做到 **40% 左右**, 已经非常接近上限 - 剩下 10% 没有覆盖的原因: 1. 一些非常 long-tail 的 question, 系统里没有现成 answer 2. Answer Quality 不够好 (freshness 或 correctness 问题)

Non-English 市场: - 由于涉及多种语言, 平均下来 Coverage 大概在 **25% 左右** - 在跨语言市场上, document 的缺失问题会更严重 - 这些数字是在 Pre-LLM 时代

LLM 阶段 (2023 - 2025): - 无论是英语还是跨语言市场都有了长足的提升 - 我们构建了 **Generative Search System**, 可以实现 **Multi-query Expansion** 和 **Iterative Search** - 之前解决不了、拿不到 grounding resource 的 query, 现在都能做到

与 Google 的 SBS 互为催化

双方互为技术推动的催化剂, 不断看谁又发现了一些新的 QnA Experience。所以 QnA 并不单纯只是 traditional text answer —— 我们也尝试 enrich answer 的形式:

1. **多媒体化:** 引入 Image 和 Video
2. **交互化:** 通过 Entity Detection 和 Linking 技术, 当用户 hover 到某个 entity 上想了解更多时, 弹出 Information Card 或 Definition Card, 快速帮用户回答问题
3. **内容的维度** —— 对于一些开放或有争议的问题, 我们做了 **Multi-Perspective QnA(多视角问答)**:
 - 用户可以查看不同的 answer, 从而 derive 一个比较 comprehensive 的观点
 - 现在有了 LLM 之后, 系统可以做更好的 summary, 并从中 distill 出 common 的部分和 difference 的部分

QnA 项目的人员规模

- **Online team in US:** 40+ 人
- **Offline team in US:** 20-30 人
- **我这边的团队 (China):** 早期 5-10 人, 后来 15-20 人, 最高时差不多 30 人
- **PM 和 Design team:** 10-15 人
- **指标团队 (Metric team):** 定义 online 和 offline metrics
- **Platform team:** 至少 20-30 人去支持 (因为 QnA 流量上升到非常高时, 接近 Bing 50-60% 流量都经过 QnA 系统)

评审委员会与中国团队的崛起

我们团队会有一个**技术评审委员会**, 要求所有 QnA 发布都必须经过这个跨团队的评审。

- 中国这边最早其实并没有人在评审委员会中
 - 一是因为时区
 - 二是因为特别早期我们刚刚加入, 也在建立信任
- 但到了**中后期**, 我们已经是关键参与者, 特别是在生命线的建模和创新方面 —— 我们基本上是走在最前面的

关键 Milestone — 领先 Google

我们的一些关键 milestone 不少都比 Google 领先:

1. **Pre-trained Model 落地:** 比 Google 早大概半年。因为我们的 strategy 是从 offline → near real-time → online 过渡,而不是直接 target online
2. **Distillation:** 我们其实在很早的时候,当蒸馏都还不是行业惯例时,就主动提出来,利用 Hinton 的蒸馏思想进行模型蒸馏。我们也是很多蒸馏前沿工作的作者和顾问 —— 比如 Multi-Teacher Distillation、Iterative Distillation、结合 RL 的蒸馏
3. **Google 也把我们当成一个 worthy competitor**

中国团队的核心贡献与产品影响力

在 QnA 项目中,中国团队在熟悉并完成 run-up 之后,扮演了非常重要的角色。特别是在 Deep Learning 模式下,基本上与算法相关、与 modeling 相关的工作,很多都是由我们去 drive 和 influence。

1. **核心贡献与产品影响力** - 我们会亲自 drive 一两个 release 来 proof concept,从而产生 product impact - 在资源有限情况下,我们通过 influence 和 fill 一些大的 team(包括推动美国团队一起 work together),完成了整个 pipeline 的 upgrade
2. **技术影响半径** - 我们的影响半径非常高 - 后期做 Near Real-Time Pipeline 和 Inference 时,我们开发的 framework 对整个 quality 产生了深远影响 - 它在 offline 和 online 之间找到了一个完美的折中,且 query 质量表现得非常 decent
3. **资源优化与成效** - 在 global 层面,我们的资源投入节省了 **80% 的 cost** - 我们利用远比英文 GPU 资源少的 resource,实现了 SBS 的 big win

无论是从 coverage 还是 quality 等各个 evaluation 维度看,这都是一次重大突破。

Research Output

- 在 QnA 领域我大概写了 **20-30 篇论文**,密度非常高
- 跨语言更是我们的一个亮点 —— 我们的跨语言 QnA 比 Google 早一到两个月发布,且第一波发布时 coverage 就达到了 decent 水平
- 我们的跨语言 QnA 在公司内部 CEO 的最高奖项中**连续两次获得 Distinguished Award** —— 这个奖项是研究团队和产品团队都可以参加的

Talent and Team

因为有很好的技术积累和很高的 performance,我那时候领导的中国团队 attracted 了很多中国顶级高校、研究所的小伙伴加入,为我们后面 scale、做更多变化的产品打下很好的基础。

这是一个非常能干、非常 elite 的 team,能够 move fast。从我们新的 AI era 往回看,我觉得这也是一个比较超前的思路:

- **人才策略:** 不是拼人数,而是拼单点的 productivity 和 creativity
 - **价值平衡:** 去 balance 大家的 immediate impact,以及我们中长期的 investment 和持续的 impact
-

故事 3: QnA System — First BERT Model Launch on Bing

Context

When BERT model was released, I caught this 信息 immediately and asked teammates to kickoff evaluation.

- **Within 2 days**, we got the results, which showed a very big uplift in quality
- My team was the first to share this within our QnA groups
- Leadership was excited because it could directly help us achieve **90% metric goals that half**

The Challenge

The challenge was clear: - **Huge resource requests** (GPU) - **Latency concerns** - Promising gains in other search areas faced the same challenge

Most teams felt the launch of the model would take long term, and bet more on model optimization by AI platform team.

My Different Approach — Knowledge Distillation

I treated this in another way. I believed that **from algo side we should do something to unblock the launch instead of just waiting**.

Inspiration: - Surveyed a bunch of papers - Got inspired by the **Knowledge Distillation** concept - This concept was proposed by Hinton before DL era, but the theory behind it sounded promising - At that moment, knowledge distillation hadn't become a common practice in industry, and no paper talked about how to leverage distillation for **model latency optimization**

My Experiment: - Conducted an experiment by myself immediately - Leveraged BERT fine-tuned model as **teacher model** to generate large-scale soft labels - Used the **online model structure** (not transformer structure, heterogeneous with BERT) for a quick training - Result: **Surprisingly 3pt F1 increase** — which was huge (original BERT achieved 5-6pt uplift) - This means we could just use an updated production model to secure **50-60% gains from the heavy BERT model with a simple try**

Quick Validation and Launch

- I quickly asked teammates to conduct more ablation study to confirm the gains
- Proposed it with our senior leads
- Because there was no extra request for any new GPU resources and no latency increase, they were very supportive for moving ahead with online experiment
- Online showed significant gains (**biggest in the past 2 years**)
- Team launched the model **within 2 weeks** — several months ahead of Google to uplift QnA quality by pre-trained models

Beyond the Initial Launch

We didn't stop here and further developed the distillation framework into more enhanced versions: - **Multi-teacher distillation** - **Iterative distillation with RL** - Etc.

Two parallel actions: 1. **Internal sharing:** I organized brownbag to share the distillation framework with partner teams inside and outside of Bing, to accelerate the adoption of BERT models for multiple products
2. **External research:** We summarized some innovative ideas and published papers to top conferences

We were actually **one of the leading forces for landing pre-trained models in production with systematic methodology**.

Recognition and Impact

Due to the super contribution to Bing products as well as the generalized impact across company: - I got **promoted within one year in Principal band** — broke the records in Microsoft STCA - The team got doubled - **2-3 core devs were fast-promoted to senior bands**

故事 4: QnA System — Multilingual / Cross-Lingual System

Situation — 为什么要做跨语言

我们 QnA 是先做英文市场。在英文市场上第三年时,我们的 model 和 pipeline 都达到了跟 Google 非常 comparable 的状态。再往下当然可以持续改进,有这样一个很好的 flywheel 包括 user feedback、iterate。

观察到的潜在 future direction:

1. Google 整个 search 不仅在英文市场,在其他市场也有发力点
2. Bing 的 revenue 在英文市场上达到比较饱和的状态。如果持续驱动 revenue,肯定要往 global 的方向发展 —— 这是一个很重要的策略
3. 从技术角度看,Bert-like 的 pre-trained model 已经起来了。虽然刚开始大家精力都还 focus 在英文,可能只有一两篇文章提到 cross-lingual training,或者很多 training 只是顺便试一下 multi-lingual 效果。但从他们 initial idea 看,这种跨语言的 transfer 能力在 pre-trained model 上已经开始涌现和显现

这几点让我有了非常强的信心 —— Bing 很有可能也会往 global 方向发展。

Inference 和 Modeling 的 Not-Ready 状态

- **Model 角度:** 只有一两篇相关文章提到 transfer 能力,但没有人知道它的 transfer 能力到底有多大、能做多好
- **Revenue 和 Long Tail:** 我们其他语言相比英文有 long tail。在 Microsoft 我们要 target **100 种 language 和 230+ 个 region**。难道要像英文一样一个一个去 scale 吗?
- **Resource 限制:** GPU 和 dev resource 不可能有大幅增长 —— 机器可能会有一些增长(因为要增加一些 market),但不会和英文同比;人基本上不会有什么太大涨幅,1-2% increase 就很不错了

Pipeline / Model 的三类潜在方向

1. **Brute Force:** 每个语言一套 model、一套 pipeline
 - 显然不 scale
 - 难道要搞 100 个 pipeline、100 套 model?Maintenance 和未来 refreshment 都是问题
2. **Language Group / Language Cluster:** 把相似 language group 起来,可以 share 一套 ranker 和 model
 - 比第一种稍微好
 - 效果和 cost 上是比较折衷,也是比较 safe 的方案
 - 仍然有不少 maintenance(即使有 3-4 个 cluster,也意味着除英文外要 maintain 3-4 个 pipeline)
 - 效果可能不是最最优
3. **更激进的做法:** 相信 transfer learning 能在语言之间达到非常好的水平,我们能 build 一套 universal model 和 universal pipeline

当时大家倾向 Option 2,但我坚持 Option 3

早期讨论中,大家觉得比较可行的是第二类方法: - 先在西语 (western language) 像 French、German 上把它们做在一起尝试 - 中文、韩文、Hindi 这些离得更远的语言,大家都非常 hesitate

当时我们参与了第二类方法的初步支持,也取得了一些可交付的成果。但在整个过程中,感觉如果要做很多集群来支撑的话成本会非常高。

随着 pretrain 的推进,跨越各种场景和任务提升迁移能力,让我觉得应该更坚定地推进第三种方法。

那时还没有官方的说法将” global”定为战略方向 —— 它更像一个孵化项目。在这种情况下,我们没有理由不去冒险尝试更理想的方向,即使当时技术水平达不到。我们应该相信,随着数据和模型能力的提升,它迟早会达到 universal 的水平,只是时间早晚的问题。

项目启动 — 高 Risk

- 业界没有现成解决方案,学术界也只有一两千篇文章进行初步探讨
- 没有人能说清楚在 100 种语言迁移后,哪些表现好,哪些不好
- 都是平均水平和一些 promising 方向,但没有哪个能在 production 中达到可用状态
- 风险来源:
 1. 缺乏可借鉴的经验(无论 research 或 industry)
 2. 不是简单 fine-tune,而是需要从根本上 (fundamentally) 进行 pretrain,增强跨语言能力
- 资源问题:
 - 当时团队没有 DGX-2 这样顶级计算模组,只有 4 卡或 8 卡 GPU
 - 很难支撑大规模数据和模型训练
 - 数据下载、清洗、存储、传输都是不小的工作量

必须找盟友 — 组建 V-Team

由于缺乏经验,完全靠自己探索可能会受限且速度慢。有没有可能邀请专家 —— 产品合作伙伴、研究团队、公司外部的专家?这对项目少走弯路非常重要。

人员配置: - 我自己挤出时间 - 抽调 2 位同事 - 但事情繁多,2 个人会非常吃力 - 必须找到盟友,组建一个**虚拟团队 (V-Team)**

三层关键合作伙伴

1. 研究院 (MSRA) NLP 团队 - 与他们建立了良好的合作和信任 - 他们当时也在寻找方向(pretrain 是热门方向但具体发力点模糊) - 我向他们力推 cross-lingual pretrain —— 在业务场景中不仅适用 QnA,也适用所有 NLP 场景,都会遇到跨语言问题 - 业务角度有很大发展潜力,research 领域也是从零到一的突破 - 基于过往信任,他很快调配了 2 位非常优秀的同事加入我们

2. Platform Team - 提供机器资源、数据存储等支持(否则我们不可能自己搭建) - 包括如何通过 NVLink 将多机训练跑起来

3. 与 Platform Lead 沟通过程 - 我与平台团队同事保持规律同步 - 当我搞定研究和这方面工作后,拿着一个简单的 proposal 去和 platform lead 沟通 - 我们要做大量 web crawl,获取/处理/清洗数据 - 当时在中国 DGX-2 这样的 GPU 还很稀缺 - 训练过程可能不稳定,需要调参、优化代码,这些都需要支持 - 当时 platform team 也在探索未来方向,只是比较通用 - 有了我们这个具体应用场景,并且他们知道我们与研究院建立了合作关系 - 虽然有

些犹豫,但他们也愿意放人进来一起尝试 - 答应借给我们 2 个 DGX-2 for 2 months(之后要看结果决定) - 同时,我也分享了 large-scale web 数据 download 和传输的 pain point - 他们觉得可以 one-off create 一个 solution 帮助我们并行去做 - 我站在他们的角度去 influence —— 这些数据本身就很有价值,除了给我们用,未来可以给更多团队用,这非常符合 platform 的 vision: **build for all, not build for one**

实验 Plan 和 Priority

因为 DGX-2 只有 2 个月使用期限,必须快速启动。

不确定性来源: 1. 数据还在收集过程中,并不是全部都 ready 2. DGX-2 随着数据量增大是否稳定?是否会有频繁 failure?需要 infra team 提供频繁 debugging 和 fix

实验规划: - 只能从 model 维度出发,确定需要做哪些实验,从最 basic 到更 aggressive 的 change - 时间上参考已有文献中在同等数据量和 GPU 类型下的 training 时间,留 buffer 估算 timeline - 目标:在这两个月看到一些答案 —— 是否值得我们持续投入 - 不仅是 platform team 的 ask,对我们也是非常重要的节点 - **充分利用好这 2 个月的时间,把最关键的 experiment 做出来** - 后续更扩展的 ablation study 可以在前面证明最基本的问题后再持续扩展 - **要有所舍弃,有明确的 priority**

整体思路:

因为之前关于多语言 pretrain 已经有一两篇文章了,我觉得要先从最基本的复现开始,跑出类似效果。在此基础上再尝试三个维度:

1. **数据层面:** 进行调整和优化
2. **模型层面:** 包括模型改动和 model size 改动
3. **训练任务:** 从 training task 角度优化

无论如何要先 setup 一个 baseline,证明 training process 是正确的。然后根据剩余时间,prioritize 数据、模型以及 pretrain task 的相关实验。

复现阶段

- 主要让多机并行能跑起来
- 验证数据清洗的有效性
- 让 training loss 等东西能在正常水平
- 这点基本上没有特别大问题,虽然中间有些小问题 back and forth 讨论,但没有特别大的 bug

在 XNLI Open Dataset 上的实验

- 当时那篇文章是在公开数据集 XNLI 上做 measurement
- 我们也跟他们选择了相同的 dataset、相同 training 数据、相同规模的 model
- 接近一个半月时,我们在 11-12 个 language 上(首先是 top language)看到了比较 comparable 的结果

遇到 Tail Language Gap 的判断

但遇到一个问题: 在一些 tail language 上一直和它们有 1-2 点 gap。

- 虽然相差不是巨大,但总感觉这里有些细节是我们 miss 的
- **时间压力:** 还有不到一个月时间,大半个月时间。如果继续在这里调,可能只是一个复现工作
- 我们的 resource 也只有那么多,没办法承担非常多 experiment

- 这里必须做一个 decision

研究院 vs 我自己的 trade-off: - 研究院小伙伴倾向于继续花时间把这两个点提上去 —— 他们觉得这个基础才比较稳固往前走 - 我觉得如果真的要 **production**,或要做研究 **target** 一个 **apple-to-apple comparison**,当然需要做这个事情 - **但因为时间很短:** - 我们也不知道在他们基础上,增加更 **diverse** 的数据、增加更大量数据,或尝试更大模型,或 **define** 更好的 **lament** 数据,是不是能带来提升 - 希望在这些维度上有结论 - **我的判断:** 除了基本复现工作之外,应该尝试一两个点的新 **idea** - 至少在 2 个月内有一些初步理解,而不仅仅是复现工作 - 加上 **top language** 看上去已经比较 **comparable**,只是几个 **tail language** 有 1-2 个小 **gap**,这个还是可以接受 - Document 下来这个 **tail gap**,2 个月后重新规划下一个计划时再去 **close**

Convince 研究团队: - 三周时间(大概)我们能够先磨合 - 现在至少能够证明我们在 **reasonable stage**,而不是完全不 **work** - 我 **convinced** 我们的 **research team** —— 他们也觉得 **make sense** - 3 周之后 **revisit** 小语种 **tail language** 的 **gap**

下一步选择 — 数据 vs 训练任务 vs Model Size

取舍:

1. **数据质量提升:** Common sense 来说,数据有提升(特别是小语种数据、跨语言数据),一般情况下这个模型肯定会得到或多或少的 **benefits**;特别量大时,有可能发生质的变化
 - 但数据 **pipeline** 才刚搭好,如果想得到量猛增的效果,这个数据还没那么 **ready**
 - 即使数据 **ready**,增加大量数据量的实验也可能存更长时间
 - 建议第一步先放掉这个 **data** 的部分
2. **跨语言训练任务 (Cross-Language Training Task):** Base on 现有数据,在数据不变情况下,**explore** 和 **define** 一些新的 **cross-language** 训练任务
 - 之前他们也没有特意为跨语言做训练优化
 - 觉得这个地方还是一个挺大的创新,可能 **ROI** 比较高
 - 如果跟 **leadership share** 时,只是强调数据增强的效果,对他们来说不会多 **exciting**
 - 但如果说有更好的方法做 **cross-lingual alignment**、**semantic alignment**,这种核心思维会更能让他们 **believe** 这个 **direction**
3. **Model Size:** 是 **nice-to-have**,因为相对简单,只需要换 **base model** 然后 **train**
 - 我们在差控的时候可以 **run** 一下,得到一些 **data points**
 - 但如果一定要 **prototype**,那增强数据之上的 **alignment pretrain task** 可能是非常重要的突破点

主要 dev resource 和 GPU resource 都放到这方面: - 很快有了大概 3-4 个增强 **cross-lingual transfer** 能力的任务 - 根据我们直觉的 **impact**,先选 2 个去做实验 - 同时在写这部分 **code** 时,我们也先跑了一个更大一点的 **model** - 之前 **train** 的是 **base model** 12 层,我们 **train** 了 16 层、24 层的 **model**,想看 **size** 的增益怎么样

三周的紧凑工作

非常紧凑,又有很强的不确定性。

- Make sure **code** 有 **cross review**,不要有低级错误
- 80% 精力放在 **new task design**
- **New task design** 在前面工作时已经有 **idea**,主要是要快速实现
- 把最强的 **dev** 放在这儿,让他能够保质保量完成

大模型训练就看能训多久算多久,优先按照 **priority** 把新的跨语言 **alignment** 实验做好。

Hit 2 个月 Milestone — 初步结果

我觉得还是挺欣慰的: - 在还不特别大量的任务上的 pretrain 过程中,加上我们 2 个跨语言 pretrain 任务 - 在各个语言上都看到相对明显的提升 - **特别是在一些相对小的语种上**(大语种比如和英文相近的语种,因为之前 pretrain 已经得到比较好的 transfer 能力)

XNLI Open 数据集结果: - 小语种 F1 score 都有将近 **5-6 点的提升** - 即使是大语种也有大概 **2 点左右的提升** - 考虑到训练时间和没有足够充分的时间调参优化,这是非常 promising 的结果

Platform Lead 的反馈:

当我们 summarize 结果跟 platform lead 讨论时,他们也觉得在这么紧张的情况下能做到这个程度: 1. 做完了一个复现,确保 pipeline 基本能力 2. 在三周时间里能够快速在少量数据上看到 trend

他觉得效率已经非常高 —— 他们决定继续让我们 keep 这些 DGX-2 的 model,继续训练。

第 4 个月 — 完整的 Ablation Study

差不多在第 4 个月时,我们已经在相对完整的数据/模型/pretrain 任务上面做了 ablation study,明显地达到了一个新的 SOTA 水平。

切到真正 Production — QnA 系统验证

XNLI 是 open source 数据集 —— 我们希望尽快在真正 production 上面 evaluate。

Start from 问答系统: - 主要做 transfer zero-shot learning —— 在英文上 fine-tune,直接到其他语言上测试 - 当时也没有其他语言,可能只有法语和德语 2 种语言少量 label 数据 - 找了一些小语种 —— 那个时候也没有大模型,所以发动了一些同事又找了额外 4-5 种语言,每种语言标了一些数据做 merit set

对比项: - 用一个没有做过多语言能力工作的 model - 我们复现的版本 model 上翻译的效果 - 增加了我们更多跨语言训练数据/训练任务/不同 model size 情况下的效果

令人 surprise 的结果: - 在一些头部 language 上,zero-shot model 已经能够很好地 extract 和 rank 我们相关的 QnA answer —— **非常 inspiring** - 在小语种上结果还不尽如人意,但在下一轮 iteration 里,从数据角度做了更多增强

第 5-6 个月 — 拿到初步 Evaluation

到第 5 个月、第 6 月初时,在 QnA 上已经拿到了 top 几个语言的初步 evaluation。

- 包括做 bug bash,让 teammates 或邀请其他同事做 bug bash
- 大家已经感觉这个 quality 看上去非常 promising

Measurement Strategy — 选 10 种 Diverse Language

- 选择 **diverse 的 10 种 language** 做 measurement
- 这也和我们 Whole Page SBS 跟 Google commit 的 language 设定一致
- 剩下没有 Whole Page SBS measurement 的语言,我们仍会去跑数据
- 只是从 optimize 角度,先把已有 measurement 的语言做好

- 这样可以先以这几个语言作为参考,跟 leadership review,让他们意识到这样的 transformation

Pitch Leadership — Move 更多 Resource

Option 3(universal solution)是完全有可能的,但要加大投入,有更多 GPU resource。

回顾团队配置: - 之前产品团队(QnA 团队)只花了 2 个 dev - 研究院 2 个小伙伴 - Platform team 2 个小伙伴 - 这就是 **core team** —— 一个非常精简的结构 - 这也符合我一贯策略 —— **Compress and Extend:** - 当 compress 时,你肯定没有那么多 resource - 你要用最小 resource 去 show tangible results - 这是一个 good timing,需要跟 leadership pitch 得到 support - 进一步 move 更多 resource,在下一年 planning 阶段也会让他们思考这是不是 right strategical direction

Right on time:

正好进入下一年 planning 阶段。在那之前我们也跟 VP review 过,他觉得这个方向肯定是非常好的方向。那个时候已经有越来越强的声音去说 Bing 的 globalization 还有整个 Microsoft 产品的全球化问题。所以我们这套 technical solution 其实是 right on time。

Distillation 的新 Topic

Review 过程中也提到 cross-language pretrained model 对 model size 要求比较高,inference cost 比较高,会比较长。

- 如果去压缩 model,它相比英文下降的比例会高很多
- 特别是在小语种上,很多效果可能就一塌糊涂
- 这引发了后续新 topic —— 如何做这种跨语言的 distillation,确保小语种效果(这是后话)

短期 / 中期 / 长期 Roadmap

那个阶段大家还没真的 move 到下一阶段,需要 step-by-step 的 roadmap。

我提出: 我们其实可以先用 —— 因为我们有三个 pipeline (online、NRT、offline) —— 先用 **offline pipeline**,针对 log 里的一些 popular query 生成 answer,进行 cache-based 的 retrieval。

Universal Pipeline 的 Coverage 问题

- Model pipeline 本身是 universal,理论上可以 serve 各个语言
- 但 evaluation 可能只在 10 个 language 上有结果
- 这种情况下应该怎么办?

我的提案: - 这 10 个 language 已经在所有小语种中占了将近一半 traffic - 剩下的 language 现在 traffic 比较小 - 我觉得 **nothing to hurt** —— 在做 flight 时,可以先 start from 每种 language、每个 market 上面的一些头部 query,生成 QnA answer,offline 建立 data store,然后进行 online lookup-based 的 triggering

建议: - 不仅是 top 10 个语言,也放一些 data 测试一下 10 个语言之外的 - 这 10 个语言都是靠 transfer,没有 specific 对它们做任何 tuning,所以它还是比较有代表性的 - 一些特别小的语种确实不是 optimal,但这个能力还在 - 相对而言,log 里的 query 也不会很多 - 其他语言也可以带上一起来飞,飞一个小 traffic,看看用户的行为和 engagement

评估方式: - 10 个语言单独看效果 (因为既有 online 又有 offline 结果) - 剩下语言 aggregate 一起看整体水平

Reasoning —— 反思维的思路: - 因为本身 traffic 也没有很高,只是 cache 一些比较头部的 query,看看用户接受程度,提供 feedback - 这对后来继续往前走会非常有帮助 - 也相信未来不可能进行 100 个语言的 movement,中小型语言可能就是一个 on-demand random movement - **Small language** 没有任何 revenue 贡献,traffic 也很小,所以 **nothing to lose**

Lead Buy-in 和 Bug Bash 检查

- 大部分 lead 觉得比较 reasonable
- 唯一就是有些 lead 心里没有底
- **我提出: 在 tail 语言里再抽查 4-5 个语言,做一个轻量 bug bash**
 - 看看在少量 query 上 defect rate 会不会特别高
 - 是不是会有非常严重的问题
 - 如果整体还不错,就大胆去飞
 - 如果真的 10 个 query 10 个都不行,那要考虑先选一些 language 进行 flight

最终结果: - 在 10 个语言的 evaluation 和采样语言的 bug bash 上都有非常 **decent** 的 **quality** - 决定直接一次性把所有语言都做一个小 **traffic** 上面的 **flight** - 在这个 traffic 上,看到聚合后各种语言表现都非常 promising

第一版 Launch 的 Strategy

这是一个 batch process pipeline,生成了 cache-based 的 QA。

- 在各个小语种上 coverage 仍然不是很高 (大概 2-3% 左右)
- 这是 intentionally 的选择:
 1. 集团计算资源有限,没有一下子投入那么多 resource 做 computation
 2. **我们也想先 intentionally 去 ship 一个小版本,先把 end-to-end 打通,在全球范围内先 launch 这个 feature**
- 之后进入了非常密集的迭代和扩展 coverage 阶段

Reflection — 整个项目的 Learning 和 Trade-off

这是 Q&A 任务,实际上我们在这个过程中沉淀了这一套框架。

后续我们又迭代进行了一系列跨语言增强训练,这一套增强训练 framework 不仅应用于 QnA,还支持了 Bing 搜索中各种 NLP task,甚至包括 Bing 之外的合作伙伴。

像 Face、News、Office、Edge 等团队都有相关 partner 主动 reach out。这已经不再需要我们主动 pitch,而是有了 visibility 和名气之后,partner 们开始主动寻求合作 —— 我觉得这是对我们工作非常好的认可。

Research 与产品双线产出

- **Research:** 最早做跨语言 pretrain 的工作,增强之后也发到了顶会 —— 是 2020 年左右为数不多的做跨语言理解和 pretrain 的工作
- **产品端:** 有非常广泛的应用
- **CTO Office 奖项:** 公司 CTO Office 组织的机器学习与人工智能优秀项目奖项评选中,连续两次获得 **Distinguished Contribution Awards**
 - 这个奖项每年只有 2-3 个项目入选
 - 我立项的关于跨语言不同阶段的工作连续两次获奖,这在 CTO 组织该活动的历史上还是第一次

- **Publication:** 发了将近 20+ 篇文章,包括 SIGKDD 和 WWW (Web Conference)的 Tutorial —— 在 research 方法论的系统性和完整性上得到广泛认可
-

故事 5: QnA System — Near-Real-Time (NRT) Pipeline

Background — 全球化进程中的瓶颈

在我们的问答系统全球化迈进过程中(包括英文市场发展的后期),我们遇到了一个很大问题:

随着 **pretrained model** 越来越强大,如何尽可能多地利用更大规模模型的能力?

- 当然可以用 distillation 等方法降低模型复杂度
- 但如果你仍然希望在跨语言场景下能有更高效率的提升,适当使用更大一点的模型还是会有非常明显的提升

早期只有两种 Pipeline 的局限

最早的 3-4 年里,QnA 只有两种 pipeline:

1. **完全 Online Real-time 实时计算** - Freshness 比较好,可以给用户带来更新的内容和体验 - 但 **online latency** 要求实在太高,基本上用的都是 **suboptimal** 的模型 - 即使成交之后,可能想要匹配原始最 strong 的模型还是会有一些 gap - 即使在测试集上看 gap 不大,但泛化到更多 tail query 上时,还是会发现它们有一些 hidden limitations
2. **完全 Offline Batch Processing** - 可以用更重的模型,但这个”重”也肯定有限制 - 主要原因: 需要从 log 里处理用户 query。在 Bing 搜索这种 traffic 下,量是非常恐怖的 —— 可能上百 million - 每个月需要 refresh 的 (包括新出现的)可能上百 million - 这要求整个 pipeline 在集中的时间有非常非常大的 GPU 量 - **稳定性问题**: 很难完全不人工干扰 - 大部分情况可以自动运行,但遇到一些 component 升级、平台 issue,可能还需人工干预 - 每次 refresh 周期比较长,可能一个月大规模只能 refresh 一次 - **没办法 target 特别 freshness 的 query**: 怎么设计哪些更快 refresh,什么慢 refresh —— 这其实也是一个挺复杂的问题

跨语言场景的 GPU Resource 限制

- GPU resource 不可能像英语那样充足,可能只有英语的 1/5
- 比如英语可能有几千张 A100 去 serve,但到了 global 上,因为 revenue 没有那么高,可能预期就是几百张 A100 甚至 V100 的 GPU 去 serve 这个 model
- 所以在这种情况下,pure 的 online 和 offline batch processing pipeline 都有非常明显的局限性

提出 Near-Real-Time Pipeline — 借鉴 Streaming Process

借鉴 general data processing pipeline 的思路 —— 在 online real-time 和 offline batch processing 之间有一种 **streaming process**。

为什么不能把 batch processing 升级成 Near Real-Time Pipeline?

优势: - 对绝大部分 (99%) query 都能得到很好的 freshness - 如果 query classification 和 query analysis 做得好,特别需要 refresh 的 query 也可以放到更高的 tier 去处理 - 因为是完全自动化、像 real-time pipeline 一直在跑,所以可以平滑波峰和波谷的 usage,全天 24 小时最大可能地用满 capacity - 不需要再像 batch pipeline 一样,在一个时间点需要集中的大量机器去做快速 refresh

这个想法比较直接,但当时 Microsoft 的 inference 并没有支持这种 small batch 或 NRT inference。

需要 Connect 的 Dots

包括上下游、Kafka Queue 等都没有 connect 起来。包括怎么从 queue 里取 request、怎么发给 inference service —— 这些其实都需要 delegate 一些 efforts 把它 build up 起来。

虽然不是巨大项目,但 connect 的 dots 非常多:

- **前端 Search Platform Team:** 把整个框架走通,从用户 request 发送到 Kafka Queue
- **Inference Team:** 把 user traffic 导过来,同时 consume queue 进行 inference service 的分发
- **Kafka Queue Team:** 是另外一个团队
- **Cache Refresh 机制:** 如果是在 TTL 内的 answer,直接读 cache。如果超过 TTL 或从未生成过,就需要快速 trigger refresh
 - 可以根据 query 的 urgent 程度,放在不同 queue 里
 - 可以有秒级、分钟级、小时级、天级的不同 frequencies 的 freshness 质量

寻找 Strong Sponsor 的过程

把这个想法快速总结成一 page,跟 platform 的小伙伴们沟通。

初期反馈: - 大家觉得很有道理,也非常 straightforward - 但刚开始没有找到 strong sponsor,或者说没得到清晰的 support timeline(因为需要 cross platform)

逐一沟通:

那段时间我跟 platform 的各个 lead 分别沟通,因为每个人都是用 platform 的一块东西。慢慢识别出了两个 **Key Person:**

1. **前端搜索平台负责人**
 - 负责把 request 发到 queue 里
 - 这个地方需要在 search platform 上做改动
 - 他发哪些东西、什么频次发、怎么送到不同的 queue 里 —— 这非常重要
 - 他从 rank-and-stack 里获取的 feature,对我们后续进一步做 QnA 也会非常有帮助
2. **Inference 负责人**
 - 负责 inference,能上下衔接 Kafka 部分
 - 以及下游如何进一步 consume queue 里的内容,再进行 small batch processing
 - **也是一个非常核心的地方**
 - 另外因为我们希望它也是一个 cache,会生成一个 cache。对于高频 query 为了节省计算量,也可以做 similar query 的 triggering —— 这部分也是在 inference 负责人的职责范围内

Convince 这两位 Key Persons

识别出两位关键人物之后,便以更高的频次找到更合适的机会与他们沟通。

- 从刚开始觉得有道理
- 到后来随着沟通深入,逐渐认识到我们目前所做的 QnA(特别是跨语言 QnA)的意义
- 以及这套方案长远来看对其他 QnA 场景的意义

- 在 Deep Learning Model Inference 和 Serving 方面,随着更多次沟通,他们也慢慢意识到了重要性
- 他们认为能以 QnA 这样重要的 feature 作为起点,在 SERP 上带来如此大增长,从零开始构建,这是一个完美的契机
- 从最初的认可,变得更加积极

我的沟通框架 —— Co-owner 而非 Ask:

这可能对他们来说不是一个 Ask,而我们是一个 **Co-owner**,需要一起推动这个 convergence ability。这也使得合作变得非常顺利,因为大家都希望项目能够顺利推进。

飞往美国 — 趁热打铁

确定方案之后已经是 10 月底,快要 11 月了。

当时担心: - 很快美国就要进入感恩节和圣诞节假期 - 这个项目原本不在他们那个 half 的计划之内 - 我们虽然得到了两位负责人的支持,但也要确保他们分配的团队成员能够及时 implement,并且对我们的系统有正确理解 - 临近假期,这仍然让人有些担忧

我的行动: - 立刻带着 2 个团队成员从北京临时出差到西雅图,待了大约 2 周 - 与 platform 团队(特别是两位关键负责人的团队成员)进行密切合作 - 基本上每天都和他们一起探讨 - 在 2 周时间里,把最重要、最核心的设计和代码 check-in 了 - 剩下的只是一些 hookup(比如 Kafka Queue 的 hookup),可能只需要半天工作量

附带价值:

我们过来一趟,还和所有 platform 合作伙伴都打了照面: - 把对他们的需求说得非常清楚 - 告诉他们,这件事情他们需要的 effort 并不是很大 - 这让大家觉得我们已经想得非常清楚了,这条路很快就能成功,而且 impact 非常大 - 大家会觉得你非常 considerate,你甚至帮他们的团队想好了怎么改代码、怎么 hookup,而且这个 effort 看起来也不大 - 所以各个团队都愿意花半天或一天的时间帮我们搞定

Implementation 时间线

- 之后还需要对 pipeline 做一些整体完善工作,以及 E2E offline 的 investment 和 stability test
- 那段时间进入 12 月底,正好也是圣诞节
- 从我们 propose 完之后,经过 11 月、12 月、1 月 —— 大约 3 个月时间,我们基本上完成了第一版的 E2E implement 和所有完整测试

Tier-wise Refresh Pipeline

第一版多语言 QnA 中,主要是用 offline batch pipeline 先生成了一些 data,把 experience 推出去,把 evaluation setup 起来。

紧接着第二波,把 NRT processing setup 起来: - 使得 batch processing data 不再是 static data - 变成了一个 tier-wise refresh pipeline - 如果是新的 query,第一次可能不会 trigger,但会送到 pipeline,后续类似 query 就会直接回答用户的 answer

跨语言市场 — 18% 的 GPU Resource

这在跨语言市场上非常有效:

我们其实只用了 18% 的 GPU Resource,就完成了 global 的 scalable launch。

这个 pipeline 至今仍然是 Bing 上 inference 的重要 pipeline 之一。

后来有了大模型之后: - Latency 和 cost 就更扛不住了 - Online 扛不住, batch 大规模处理 query 也不现实 - 所以这种 NRT 就成了唯一选择 —— 可以每时每刻 process、refresh, 无论 freshness 还是 quality 都有保证

成为 Platform Team 的明星项目

这个项目成了当年 Platform Team 的明星项目。

它并不是重新造了很多轮子, 而是以 QnA 作为契机, 把 Platform Team 散落的 7-8 个 pieces 在一个场景中完美地呈现并 connect 在一起。

这是之前 Platform Team 想做但一直没人能推成的一个问题, 结果让我们一个外部 team 以这种形式体现出来了。

CVP 的 Appreciation

- 当时整个 org 的 CVP 非常 appreciate 我们去推动这个事情
- 更重要的是, 我们是一个 remote team, 核心老板都在美国, 时区沟通其实并不 friendly
- 但正是靠着我们的坚持和持续不懈的 influence, 让大家一起成就了这个 win-win 的 story
- 我们的项目在 Platform 年度 All Hands Meeting 上成为了唯一一个对外邀请的 demo 项目
- 整个 platform team 都非常 exciting, 因为他们看到了自己的 investment 放在一起发挥的效果
- 我们也向他们展示了上线后整个 global QnA 带来的 metrics (类似 Side-by-Side)
- 当时我们的 coverage 和 freshness 都是领先 Google 的

Pipeline 的可复用性

这套系统是可复用的。除了 QnA, 任何具有以下特征的场景都可以使用:

1. **Dependent on heavy models** (有一定的 latency 和 freshness 需求)
2. **数据量巨大, 需要持续更新**

后来 Microsoft 开始投入 **Feeds Recommendation**: - 涉及 Feeds 的 Content Index 生成和 Featurization - 当时对于深度模型的 NRT inference 基本没有支持 - 有了这条 pipeline 之后, 后续做 Feed 时就非常自然地拿过去做 NRT 的 content 和 feature generation - 整个 Feeds 产品线都因此受益

如果没有早期推动, 后续很多事情的进展可能都会受到 delay。包括我自己后续做 Knowledge Card、Underside Feeds、GenSERP 等等。

Reflection — 几个非常关键的点

回顾这个作为 remote team 去 propose idea、细化方案、influence 8 个 org 外的 partners (跨时区)、最终落地的过程, 我觉得有几点非常关键:

1. **主动出击, 化被动为主动** - 我没有坐在那等 Platform Team 帮我 figure out - 理论上这是他们的 mission 和专长, 但他们去做这件事的 motivation 并没有那么强 - 在这种情况下, 我们要有学习精神, 尽可能去 connect the dots 并 get clarity - 当你把一个更可行、更具体的方案摆在他们面前时, 对方能感受到诚意 - 他能清晰地量化自己的工作 (上游、下游、工作量是一天还是一周), 这样他才知道是否可以以最高 priority 来 support 你

2. 找到关键的 Partner - 项目中合作的 partner 至少有 8 个,但深入沟通后你会发现,其中只有 **1-2 个是核心** - 只要把这几个核心点走通,其他的点就会迎刃而解 - 在一个项目中不断消除 ambiguity,并快速 build up 框架去 influence 别人 —— 这是非常重要的方案

3. Bias for Action,趁热打铁 - 如果不赶紧把关键技术点打通,可能圣诞节、感恩节一放假,2 个月就拖过去了 - 在这种情况下,我们不可能在当年的 fiscal year 完成 launch,更不可能赶在 Google 之前推出去 - 只有及时占住基本盘,后面才能腾出更大的心理空间去做 iteration

不管是技术本身,还是这些 learning,在我后面的很多项目上都得到了应用。

故事 6: Knowledge Card — Interesting Facts (2021)

项目 Context

Knowledge Card 是 Bing 从 **2021 年**开始推出的一个新的 Knowledge Experience。

原先的 **Entity Pane**: - 无论 Bing 还是 Google,在 SERP 右侧的 Right Rail 都会有一个所谓的 Entity Pane - 以前的 Entity Pane 无论从 UX 还是 Content 上都比较平淡 - 更多用于 **Knowledge Discovery**(知识获取) - 比如查询一个名人的身高、体重、年龄,用户虽然有收获但没有更强的 engage 意愿

Knowledge Card 是完全换新的升级版本,在两个维度突破:

突破 1 — 内容维度 (Content)

除了提供基本 fact 以及传统结构化的 knowledge 之外,更希望挖掘一些非结构化的、更具吸引力的 facts:

(a) Intriguing Facts —— 这是一种非结构化内容 - 不像传统的”主-谓-宾”(Subject-Object-Predicate)三元组那样只提供中规中矩的数据(如演员的身高) - 而是以 **Narrative**(叙述性)和 **Natural Language**(自然语言)的形式呈现 - 具有更强的容量和更丰富的内涵,让用户产生更强的参与感

(b) 丰富的交互内容 —— 除了 Intriguing Facts,还引入了: - **Intriguing Comparison** - **Intriguing Poll**

很多 entity 具有争议性话题,通过这些功能可以让用户参与表达自己的想法,同时查看他人的观点。这些内容都超越了我们传统的 **Knowledge Graph**。

突破 2 — 体验维度 (Experience / UX)

(a) 视觉升级: - 采用更加 Vivid 的 3D Image,或更大、更 prominent 的图片 - 视觉冲击力更强

(b) 配色呼应: - 整体配色能够与 entity 主题以及整个 SERP 配色相呼应 - 相比以前 Entity Pane 那种”黑白电视”般的单调感,Knowledge Card 给人一种”彩色电视”般的丰富观感

项目背景 — 两个团队失败的尝试

从 2020 年开始团队就在这个方向发力。在我正式参与之前,已经有两个团队前前后后持续探索了将近一年时间做相关尝试。

关于 UX 的挑战不在介绍重点中 —— 当时我的 team 还没有做 UX,所以参与这个项目时,主要是支持 intriguing content,特别是最早的 intriguing facts。

例子: - 章鱼的血液是蓝色的 - 长颈鹿是站着睡觉的 - 火星上有一个山,它的高度是珠峰的 3 倍那么高

为什么这些 facts 特别: - 你以前的 knowledge base —— 如果拆成三元组会非常晦涩,体现不出有趣 - 或者根本没办法用已有三元组形式很直观地表现出来

用户调研的发现: - 产品团队做了一些调研,发现这部分非常大的潜力 - 小视频等等都是在传递一些给用户更多信息、传递更多用户未知信息,让用户能够有更高兴趣去点击 - 我们希望在传统 search 产品上,能有一些更加贴近用户的 exploration knowledge 体验

Coverage Goal — 80% vs 16%

两个团队失败的现状: - Leadership 期望: 在头部 entity 上达到 **80%-85% Coverage** - 之前两个团队卡在 **16%** 左右 - 如果 coverage 质量再卡严一点,量就更少 - 所以他们基本上都要放弃了

我加入项目的契机

美国 Leader 的态度:

美国 leader 说”有这么一个事情,如果感兴趣可以看一看”。也希望我们能贡献和推动。他刚开始的时候预期并没有那么高 —— 大概是”帮忙看看已有 pipeline 有什么问题,看看能不能帮到多少”这样一个态度。

我的判断 — 不会有 Low-Hanging Fruit

但我觉得就是说,两个团队的 engineer 在这已经做了一段时间了,肯定不会有 what low hanging fruits 就躺在那让你去捡。

那肯定是有 some reason 它就卡住了,没有踩到真正的点,没有找到真正的方向。

所以我在真正去 propose 任何东西或 idea 之前,还是花了一周多时间去详细看他们的 design docs、pipeline,做一些数据分析。

短暂陷入误区 —— 然后跳出

我当时其实也短暂地陷入了一个误区。

- 最初想的也是,是不是帮他们在已有 pipeline 上做一些修正?他们也有方法去做一些修改
- 但我很快觉得我应该更重要的是去理清楚,搞明白它现在的 bottleneck 到底在什么地方

一周分析后找到的两个根本问题

问题 1: Pattern-Based 的 Extract Fact 方式 - 它们的 Extract Fact 时,可能很多是 pattern-based 方式 - 我们在 search engine 中有这么多种类的 entity,对应的 document 样子也非常 diverse - 如果是靠它们的话,你肯定会有遗漏,肯定是不 scale 的 - 还要考虑跨语言 —— 难道要把所有 pattern 都翻译成各种语言吗?想想脑袋都有点大

问题 2: 把 Interesting Facts 抽象成 QnA 问题

两个团队都把这个问题抽象成了一个 QnA 问题。他们的做法: - 对于 target entity,先从 Bing 的 search log 里找到和这些 entity 相关的 query - 用这些 query 把 query 发到 QnA 系统里 - 因为 QnA 系统会做这种从 passage 里、从相关 document 里去抽 passage,把和 query 最相关的 passage 抽取出来 - 抽取出来的其实和我们想要找的 fact 基本上就比较接近了 - 所以他们希望通过 reuse QnA 系统去做这个事情

我觉得这实际上是一个很 smart 的 idea —— **leverage** 我们已经有了成熟的系统,然后去快速推进这个问题。

我刚开始觉得也没有什么问题,大概那一周我也是 buy in 把它 frame 成一个 QnA 问题的做法。

Framing 错误的洞察

但随着我看越来越多的数据(包括 PM 给的一些数据),我就越来越觉得 —— 突然意识到把它 frame 成一个 QnA 问题,实际上本身这个 framing 可能就是有问题的。

举例 —— 火星山的 case:

火星上有一个山,它的高度是珠峰的 3 倍还要高。

这个 case 的问题: 1. 首先很少有人知道这个 fact。如果不知道的话,用户也很难去问自己不知道的事情 2. 即使用户问这个东西,他能问到的和火星相关的问题,然后能用这个 fact 去 serve 的概率也未必很高 3. 英语上如此,那到了小语种就更是这样了 —— Bing 的小语种非英语 traffic 比英语少很多(整个非英语加在一起可能才抵得上英文的 traffic)

关键洞察: - 这种情况下,和 entity 相关的 query 又能问到这个 fact、能用这个 fact 去 serve 的 query,那就更少了 - 从这一点上,query 本身就做了一个很强有力的 pre-filter,把很多 fact 挡在了门外 - 不管你的 ranking 后面的逻辑做得多么好,你可能都在 recall 这一层就已经……

理清清楚这一点之后,我就非常明确地意识到我们应该修改我们的 framing,要去掉这个 query 的 dependency,从 entity 直接建立和 fact 的联系。

我的新方案 —— Document-Based Approach

- **Query-based approach** (旧): 给定 entity,先找 query,通过 query 去抽 fact
- **Document-based approach** (新):
 - 给定 entity,不需要找 query,而是想办法直接 link 到这个 entity 相关的文档
 - 然后从这个文档里用一个 **document level** 的机器阅读模型 (**MRC model**),去对这个 entity 相关的 fact 进行抽取
 - 进一步根据 feature 进行 ranking 和 selection
- 这其实还是一个蛮大的改变

快速搭建 Demo

为了验证可行性,我们很快就搭了一个 demo。比较好的是我们团队原来其实做了非常多年的 QnA 系统,所以积累了很多各种各样的任务模型: - 比如这种 document level 的 MRC model,以前也是用来抽文章中的 passage 的 - 只是 passage 可能会更长一点(大概 2-3 句话),但我们这里的 fact 可能基本上都是 1-2 句话 - 这种情况下,对我们的模型快速做一些微调,在 decoding 时做一些微调就可以

核心模型组合 (Minimum Effort): 1. 想办法把 entity 和 document 连接起来 —— 原来本身就有很多相关工作(比如 entity stamping 等),所以这块有一些前续工作可以依赖 2. 怎么从这个 document 里去抽 fact —— 而且抽出来的格式各方面都是好的: - 独立存在的话,感觉也是 self-content 的 - 不能说“它怎么样它怎么样” —— 因为只有 1-2 句话,如果不是 self-content 的可能会带来 confusion - 所以这个要求还是蛮高的,但还是专注于怎么把基本的 recall 做好

第一次跑就 70% Recall

我们搭建了之后,在一个比较小的 query set 上面,大概有 500 个 query: - 这也是前面两个团队他们用的 marmot set,我们就直接拿它来跑 - 第一次跑没有做太多 tuning,recall 基本上可以达到 70% 的水平 - 我们快速 eyeball 了一些 case,发现 candidates(比如刚才那个火星的例子)它都很好地召回了

有了这个之后,下一步就很直接: - Define 各种各样的 feature 和 ranking model,把好的结果 boost 出来 - 去掉那些不好的结果 - 解决 recall 的问题真的是解决了一半的问题

VP 的反应

把 preliminary 的结果和 design 总结之后 share 给我们的 VP,VP 非常非常激动: - 因为这个问题已经很久没有听到新的声音了 - 包括我在前面第一周里其实也 buy in 了这个 query-based 的方法 —— 当时想的就是“是不是 query 的生成和 mining 要改造一下,是不是 QnA 系统的抽取和 ranking 要改造一下” - 后来发现完全不是这么回事 - 它真正的问题并不是算法本身,更重要的是你怎么去 frame 这个事情

关键 Reflection — Framing Matters

如果你从一开始的 frame 就发生错误了,那你后面的方法就会有一个非常有限的 upper bound。

这种思路和思维其实对我后面的很多项目都带来了很深刻的影响。

特别是当你接一个新的项目时,再去看待你不熟悉的事情时,不要默认这个东西就是理所当然应该怎样的,而是要有某种批判的眼光去思考:

首先它基于的 framing 就是正确的吗?如果说它的 framing 这个方向本来就是错的,那其实它后面的方法收效也就会非常的微弱。

后续的迭代和最终 Coverage

经过多轮 iteration 把这个 pipeline 完善之后: - 轻松达到了 85% 的 coverage - 甚至在更头部的 entity 上达到了 95% 以上 - 这保证了 Knowledge Card 项目具有真正的价值,也为后续 enrichment 和更多 feature 打下了很好的基础

第二个挑战 — Interestingness 评测

Recall 问题解决得差不多之后,我们立刻遇到了关于 Interestingness(趣味性)定义的问题。

Interestingness 是一个比较主观的指标,很难去定义。

那会儿大模型还没有出现(离大模型出来还有半年左右时间)。我们当时采取的方案:

1. 评价体系的演进:

(a) Point-wise Judge (最早期阶段): - 主要判断 fact 是否 appealing - 比较容易区分特别有趣和特别没意思的内容,可以把表现最差的干掉 - 但如果想做更精细的排序,这种方式就显得比较 random

(b) Pairwise Comparison: - 为了更精细地比较,我们开始做 pairwise

(c) Iterative Hill Climbing: - 在多个 candidates 的情况下,通过迭代竞选出最好的一个或多个

2. 模拟用户评价: - 我们利用 Crowdsourcing(众筹/众测)或 Consultant Judge 来模拟用户 - 对这些 fact 进行评价

大模型出现后 —— Diverse Role Play

等到半年后大模型出来,给我们带来了机会。

从内容生成的角度: - 抽取更准: 大模型可以发挥更强的抽取能力 - **改写与消歧:** 针对 format 有问题、不是 self-contained 或存在指代不清的内容,进行适当的改写和消歧,使其达到用户能够理解的程度

Diverse Role Play 方法 —— 利用大模型做”可主观评价”的天然优势

我们提出了一种 **Diverse Role Play** 的方法。

核心逻辑:

1. **多样化模拟 (Diverse Role Play):**
 - 即使对于同一个 fact(事实),我们可以让大模型模拟不同的 **persona**(人格特质)
 - 从而对该事实的有趣程度进行多维度评估
 - **通过多种 persona 的模拟,大模型能够真实反映出不同人类群体在阅读该内容时的反应**
2. **兼顾多样性的聚合 (Aggregation):**
 - 在进一步做数据聚合时
 - 这种方法能兼顾不同的 diversity(多样性)
 - 从而让排序 (rank) 效果更出色
3. **个性化分发 (Personalization):**
 - 我们可以根据当前用户的 profile,将真实用户映射到某个特定的 persona 上
 - 再把该 persona 对应的评价最高的 fact 推荐给用户
 - **这是一种非常高效且有效的个性化方法**

业务提升: - 这个方法对我们业务的提升非常显著 - 帮助我们极大地提升了内容吸引力 (Interestingness) 和用户参与度 (Engagement) - 在 **Top-line Metrics** 上至少提升了 **10%-15%** 以上

Agent vs Human Reviewer 对比

相比之下,Agent 的优势比人工(如 Crowdsourcing)更加明显:

- **人工众测:** 你无法准确获知测评者的背景,也很难控制背景的多样性和分布
- **Agent 优势:** 它是高度可控的 (Controllable)

总结来说: **Controllable agent is better than non-aligned human reviewer.**

第三个挑战 — 量的问题 + Agile V-Team

另一个挑战在于”量”的问题。因为 Bing 上有非常大量且 diverse 的 entity。

- 这些数据不需要实时 (real-time) 生成
- 可以利用预先构建一个 entity 的 fact index 这种 data store
- 但如果使用 batch 处理,会对 GPU 产生巨大的算力需求,同时 freshness 也会出问题

复用之前的 Global QnA NRT Pipeline: - 我们之前在做 Global QnA 时,已经建立了一套非常完善的 QnA solution - 因此可以进行一些调整和适配,将其应用到当前场景

没有正式 Resource — Agile V-Team

这是因为我们在 Knowledge Card 上(特别是 Engaging Facts 方面)取得了一个比较突破的进展,得到了 VP 的高度支持。

在做这件事情时,我们并没有被正式分配该项目的 resource。我其实是挤出了一个 Dev 去做一个快速的 prototype,并鼓动我们的 PM —— 三个人组成了一个非常小的 V-team。

那会儿我们既没有正式的 productivity 任务,也没有小模型的支撑。但我认为这种 Agile 的 V-team 模式,去 prove concept 并不断迭代的思维已经非常成熟了。

我们只是在整个团队里挤出资源去推进这个 prototype,最终得到了美国团队 VP 的认可。

Resource 投入与团队转型

接下来的进展:

1. 资源投入与交付: - 因为这个 team 展现出了非常强的 delivery 能力 - VP 觉得应该给这个 team 更多的 resource
2. 核心诉求与转型 —— Full Stack: - 在这个节点上,我也适时表达了诉求 - 跟美国团队合作多年,拥有 end-to-end 的 experience - 我们其实一直专注于做模型、数据和 backend pipeline - 但有些时候,我们一些好的 idea 仅靠 backend 和 model 是无法完全实现的,需要结合前端 UX 才能有更好的展现形式 - 我一直希望把团队的 skill set 转型为 Full Stack: - (a) 保留我们最擅长的 Model、数据和后端 Service - (b) 引入前端设计能力 - (c) 在一个 Team 里把这些环节整合起来

Knowledge Card 里有几个关键的人,基本上也是那几个 Principal。

3. 效率提升: - 我们的 VP 和 Senior Leadership 非常支持这一努力 - 他们也觉得我们提出来的诉求非常合理

人才储备的提前布局

为了这件事情,我其实做了挺长时间的准备,尤其是在人才储备方面。

- 因为我毕竟是 Machine Learning 和 Applied Science 背景出身
- 在 UX 上虽然有一些自己的 taste,但要说系统性的理解和实现,确实还没有太多经验
- 所以肯定要找一个比较强的 lead 来负责
- 因为一直有这个想法,所以在这之前,我已经以另外一个项目的名义招进来了一个 Lead
- 正好趁着现在的契机,加之老板们也非常 supportive,我之前储备的人才就派上了用场

Lead 的快速加入

在正式开始这个项目之前,他已经在我的 team 里有了一些 run-up,了解了很多基本的工具和做事方式。

当他转到 LogicCard(应该是 Knowledge Card)这个项目上时,基本上可以全力以赴、Full speed 地去 push。

团队 Excitement 和 Communication Cost 降低

团队里的每个小伙伴做这个项目都觉得非常 exciting,因为它既延续了我们的 strength,同时又有很多新的东西。所以大家都非常有干劲。

这样可以显著减少 **communication cost**: - 一些小的 feature 或问题,我们可以在内部 in the loop 快速解决 - 而不需要每次都去协调美国 UX 团队的精力

在 VP 心目中地位的提升

这次我们解决了一个非常”烫手”且 challenging 的 problem,在 VP 心目中的地位有了非常大的提升。

虽然他们以前也很 trust 我们,但这个特定的问题之前 **slow down** 了非常多的时间,没有人质疑它 **framing** 的问题,导致一直没找到突破点。

我帮团队找到了这个突破点并且给出了 landing solution,不仅 **save** 了 Knowledge Card 项目,还让它从一个 **lowlight** 变成了 **highlight**。

有了数据的支撑去做 experience,Knowledge Card 更加大放异彩。它对用户来说不再是华而不实的东西,而是既有内容又有颜值的 **feature**。

Knowledge Card → Knowledge Card → ORCA 平台的过渡

我差不多是在 2022 年左右开始接手这个项目的。

- 其实在 2020 年前,QU (Query Understanding) 平台的建设就已经在筹备中,并做了一些 pilot projects
- 直到 2021 年底、2022 年初,我们正式成立了一个 Universal NLU (ORCA) 团队

之所以做这个项目,一方面是因为在 Bing 内部有很多 vertical 的 answer 以及 segment experience,它们都需要对 query 端做 triggering,来决定:

1. 这个 Answer 是否满足显示的条件?
2. 满足条件的情况下,它的质量又如何?

(过渡到下一个故事 ORCA)

故事 7: Universal Query/Document Understanding Platform — ORCA

项目定位

ORCA 被定位为一个 **Universal**、跨语言且多语言的 Query Understanding 和 Document Understanding 服务平台。

为什么发起 ORCA — 主要原因

1. **统一技术路径**: - 各个 Team 和 Segment 在处理 Query Intent 以及各种 Classification 时,做法千差万别 - 这种碎片化的状态非常难 **convince** 合作伙伴,也很难维护 - 我常跟同事说 —— 还是要保持沟通渠道的畅通,尽量维护团队的平稳,否则会对整个 Team 产生负面影响
2. **成本与效率 (Budget 角度)**: - 从预算和资源角度,公司不希望每个团队都从头训练一个模型再去做 **distill** - 像我们当时的情况,因为项目已经在赶进度(在路上),根本也来不及去做 **distill**

ORCA 平台的意义 — 一条龙服务

关于 ORCA 平台的意义,最重要的一点是我们可以帮助更多的 Partner Team(无论是否有相关背景)快速 onboard。

我们提供从 Train(包括)到 Serve 再到 Monitor 的”一条龙服务”,是一个 Highly Leveraged 的项目。

经验复用 — From QnA Cross-Lingual Learnings

- 这个项目借鉴并参考了我们在 QnA 领域的跨语言理解经验
- 我们当时 pretrain 的模型,后来也被其他团队拿去针对自己的任务做 fine-tuning(微调)
- 这正是我们预期的状态
- 大家开始使用 AI 去探索,将自己武装成 multi-talented 的角色,这本身是一件好事

Customer Obsession — 给 Internal Developer 用的产品

- 这其中也体现了 Customer Obsession(客户至上)
 - 因为我们要给内部的 Developer 使用
 - 所以产品的 Dashboard 以及如何切换 Full Screen(整体状态与局部状态的切换)等交互细节
 - 如果有精力最好去深思熟虑一下,因为这非常考验我们的 Productivity
-

故事 8: Undersider — Bing × Edge Right-Rail Content Pipeline

项目概述

Undersider 是 Edge 浏览器上的 right-rail AI companion。当用户选择 expand 时,可以提供: - AI Keyword (帮助用户更快消费当前页面信息) - Suggestions for further exploration

项目意图: 这是用户在浏览过程中非常重要的一个 companion —— 帮助用户更快找到新的信息、完成任务、并促进进一步 exploration。

项目背景 — Edge × Bing 的特殊合作

在 Undersider 之前的 Edge: - Edge 浏览器团队是一个纯粹的 Software 团队 - 基本上没有 Machine Learning 团队,主要做 Feature 和 Software - Edge 团队是和 Bing Core 团队平行的一个很大的组织

为什么这个合作天然成立: - Edge 是 Bing 非常重要的流量入口 —— Bing 上面 1/3 甚至更高的流量都是通过 Edge 过来的 - 帮助 Edge 更好地服务用户 = 增加用户 retention = 帮助 Bing 持续增长 - Edge 需要 Bing 提供积累的大量 DL Model 和 Data 来 bootstrap 整个 Undersider - Bing 也希望让 Edge 用户和 Edge 有更强的联系 —— 增加 Edge 应用时长,带给 Bing 更大可能性

基于这个 win-win,我们成立了与 Edge 团队一起合作的 V-Team。Project Code Name 就叫做 Undersider。

项目复杂度 — 跨部门合作

技术部分不多讲,因为很多技术是从 Bing 场景中积累出来的。这块比较复杂的是跨部门合作,因为 Edge 是一个相对而言比较垂直和传统的团队。

挑战 1: Edge 的 Shipping Cycle 与 AI 时代的不匹配

- Edge 的 shipping cycle 非常长 —— 可能一年就两到三次发版
- 你只能 catch up 两到三次发版,然后 ship 东西
- 但这显然在 AI 时代和我们 model 快速迭代是格格不入的

怎么 influence Edge 团队?

平衡两件事: - **软件稳定性** —— 给用户 deliver right message,不能频繁大改让今天和明天发生巨大区别。这对用户也不是好的体验 - **快速迭代的需求** —— 一定程度上要兼顾 AI 时代的 model iteration

Process-wise 节奏的调整: - 产品迭代的周期 process-wise,需要跟 Edge 团队磨合 - 磨合不是一蹴而就的 —— 一步一步建立 trust,然后用 Bing 里面快速迭代的观念去引导他们 - 制定一个比较 reasonable 的节奏和流程 - 从最初 follow 他们的 quarterly 周期 → convince 他们到 monthly 节奏 - 通过不同 environment 更快速地测试,refine,catch up,monitor,train,launch

挑战 2: 作为 Remote 团队的 Influence Without Being In The Room

- 作为 remote 团队的 leader,这里面有非常 complex 的 conversation
- 我在 remote 团队,Edge 团队 leadership 主要在美国
- 很多会议我也不在 room 里面
- 我更需要在美国找到 **Engineer Leads** 和 **PM Leads** 的同盟 (allies):
 - 他们可以把一些想法在我参加不了的 meeting 里面 deliver
 - 然后能够跟 Edge leadership 达成决定

这在合作中是非常重要的部分 —— 你怎么能够 influence 那些你不在的会议,然后让它能够往你预期的方向去走。

总结两个 Influence 维度

1. 和 Edge 团队不断 earn trust,优化他们的流程

2. 从 Leader 的 Remote 团队角度,在参加不了的会议里有 influence: - 把技术做得 solid,prove 好的 idea(基础层) - 在这个场景里更重要的是 **communication** - 在很多我们不在的会议里,非常 dependent on PM 和我们的 leads - 推进需要有自己的 ally 的支持

挑战 3: Resource — High-Leverage Setup

并行有 3 个大的 area: - 那个时候在做 **Knowledge Card** (QnA 已经进入 good state,只少量 resource maintain) - **Knowledge Experience** (Knowledge Card) - **NLU Platform** (持续在做) - 加上 **Undersider**

资源竞争状态: - Undersider 是一个非常大的项目 - 我们把整个团队都投入进去可能都不够 - 当时的策略 —— 用少量 resource 发挥大的影响力: - 我们做 Undersider 只有 10-12 个人(整个 Undersider 项目加在一起 100+ 人) - 想办法 influence、负责一些 ROI 比较高的事情,达到杠杆效应

两大 High-Leverage Focus

1. **内容生成 Pipeline** - 团队积累了非常多 NRT 内容生成 pipeline (从 QnA 到 Knowledge) - 在 MSI 里非常重要的一部分就是内容生成: - 给用户推荐问题 - 帮助用户总结刚才的内容 - 这些都需要内容生成来支持,填充内容索引 - 与其做非常多零散的 feature,不如**推动和负责内容生成 pipeline** - 同时为了知道 pipeline 目前缺陷和

问题,也负责一两个比较轻量的应用作为抓手: - 页面的 Instruction Generation - 关于当前页面的 Topic Detection - 这两个应用场景能让我们亲力亲为做事情,主要目标还是把内容生成 pipeline 做好,支持所有 Undersider 的项目

2. 个性化推荐算法 (Phase / Personalization) - 早期主要这两块:内容生成 + Phase - 基于当前页面给用户推荐相关 document —— 是一个 phase 流,需要根据用户 interest 做 presentation - Bing Team 之前积累比较少 —— 因为 consumer 测试场景下更 care code relevance,presentation 不是主要目标 - 这是一个需要去探索的事情 - 我们有 5-6 个小伙伴负责做 Personalization 算法

Backend 和 Experience 由我们美国团队合作。这个 Phase 最后是 NLU 里面一个 horizontal feature —— 在 NLU 里面有非常高的、非常好用的 feature。

内容 Pipeline + Recommendation 的 Phase 算法 = 给团队定位的 high-leverage setup。

Phase 2 — Edge Copilot 上的长文档理解

后来 Undersider 这块,有了大模型之后,我们也开始做 Edge 上的 Copilot。

时间线: 2023 年初 (大概 2023 年 4-5 月份),Edge 上已经 launch 了第一版 Copilot。

关键能力缺失 —— 长文本理解的支持: - 主要受限于大模型的 token limit - 它无法处理特别长的文本 - 在 Edge 上,用户已经打开了一个 page。当 page 特别长(比如 long PDF): - 大模型可能没有足够 context - 让它做 summary 或其他操作 → 可能出现非常严重的问题或不能回答用户

那时候还没有 Long Context 大模型,需要 figure out 方法。

美国团队的初始方案 vs 我的提案

美国团队的两种 baseline 做法:

1. 只取 context window 范围之内的内容,剩下全部舍弃 —— Baseline
2. 把文章分成多片大 chunk —— 默认是第一个大 chunk,如果第一个 chunk 找不到内容,会依次去查看其他 chunks

让大模型当然是可以 workable 的,让它看不同部分,然后再 aggregate。这当然可以,但 cost 和 latency 都是比较大的问题。

我的方案 — 多级 Index Retrieval

我们团队原来做 QnA 积累了很多这样的经验。其实可以 build 一个快速的 Index —— 每个 document 都可以 build 一个快速的 index,根据当前 query 快速定位到片段。这个过程可以用更轻量的 model,而不是 large model 那样复杂去增加这么多 cost。

多级 Index 设计: - 三级 index:Segmentation → Paragraph → Sentence - 也会有 Embedding 的 index:整个 document 的 embedding + 各个 segmentation 的 embedding - Query 来了之后,用 retrieval model retrieve 相关 segment,甚至定位到 paragraph / sentence level

优化目标的不同: - 这部分相对于以往场景 —— 更重要的是把相关的 recall 做好,把尽可能相关的 context 都找全 - 优化目标和以往 search 很不一样 —— 它要把 recall 尽可能做高 - 后续可以依赖目前的能力做进一步筛选,因为把这个东西作为 context 给大模型时,它还是会有很强的判断力

Adoption 路径 — 通过印度 Team 的 PDF 突破

提出方案后,基于之前的模型搭建了一个 prototype。

Adoption 不是很 smooth: - 我们在 Edge 前端和 client 端的改动没有经验 - 必须尽快找到 partner,把初步 prototype 做起来 - 但他们也有别的一些 party,包括美国团队也在思考怎么解决这两个大方面的问题 - 你要说我们是更 promising 的方法,其实并没有那么容易

Low-Effort Demo 策略: - 一边还在 commit - 另一方面通过 low effort demo 去 demonstrate capability

关键洞察 — PDF 切入点:

在沟通过程中,我发现 Edge Browser 里面有一类非常有代表性的 document 就是 PDF。负责做 Edge PDF 的 AdSense Team 实际上在印度。这个印度 Team 也在思考怎么能让他们的 PDF Segment 里面的 user engagement 更高,甚至当用户用 Edge 打开 PDF 的时候,能够让用户发现更智能化的能力。

所以 adoption 顺序就清楚了: - 把这个 Long Document 能力 scale 到所有 web document 之前,可以 start from PDF 先去把 capability 展示出来 - 同步跟印度 team 讨论 —— 他们觉得这个能力非常好,愿意 support 我们做一个 project - 如果能在 long PDF 上证明这件事,实际上反过来去说服美国团队的时候,他们应该会欢迎 —— 因为 scale 到更多 web document 上是 natural extension

Result

事实证明,确实如预期: - 很快在 PDF 类文档上做了一个端到端 prototype - 展示了非常出色的长 PDF 阅读理解能力 - 比如像一两百页的 PDF,用户可以在长内容中任意提问 —— 立即就和现在的 baseline 产生差异 - 向美国 VP demo 时,他觉得非常 impressive - 按照计划,我们应该从 PDF 扩展到 Web Document - 在 VP support 下,非常 naturally 得到核心团队的支持,开始从 PDF scale 到更多 documents

Reflection — 多个 Reusable Methods

- 1. Influence without authority over a vertical, slow-cycle partner team:** - 用 trust + ally network + process-wise 节奏调整,把对方的 quarterly cycle 牵引到 monthly cycle - **Don't fight legacy culture; ride alongside it until your value compounds**
 - 2. As remote leader, allies do the work in rooms you can't enter:** - 找到 Engineer Leads + PM Leads 的同盟 - Solid technique 是基础,但 **the multiplier is communication**
 - 3. High-leverage focus when resources are 10x smaller than the project:** - 不做零散 feature,negotiate 自己 own 一个 horizontal capability (内容生成 pipeline + personalization phase) - 这样所有其他 sub-project 都来 leverage 你
 - 4. When your method needs adoption, find the underdog partner first:** - 印度 PDF AdSense Team 比美国主线 team 更 hungry for innovation - **From their adoption you earn the leverage to convert the bigger partner**
-

故事 9: Generative SERP / Copilot Search

项目 Background

2023 年,Microsoft 是第一个率先用大模型去做 Search 入口的。

当时的做法: - 把它标注成一个 **Vertical** —— 这种方式 risk 比较小 - 重新有一个 vertical 去做这个事情,减少对已有业务的影响 - 但其实它的 **Growth** 也慢慢遇到了问题

2023 下半年 — 关键 Direction Pivot

我们觉得应该要利用 LLM 的能力去重新 **reinvent** 我们的 **Traditional SERP**。它应该兼顾 conversational 的能力,用户可以问更复杂的 natural language query。同时,query 也能 benefit from traditional SERP rich 的 format 和 content。

当时和美国 Lead 一起提出新概念 —— **Generative SERP**:

- 直接在 SERP 上进行革新
- 因为 SERP 上面已经有足够高的用户 traffic
- 与其去做各种 promotion,不如让 **SERP experience** 做到更加 modern、超前
- 让用户不离开已有的 **SERP** 就能感受到大模型的能力
- 觉得这是搜索引擎必然会走到一个阶段

团队职责分工

我的 team 主要负责两大部分:

1. **Content Quality** —— 基本上 100% 是中国团队去做的

2. **UX Data** —— 我也有一部分 UX Data 可以 support 美国 Team 关于 UX 的一些 innovation

Setup 是 Complementary 的: - 美国 Partner Team 主要以 **Engineering** 为主 (Backend Service / 前端等) - 我的 Team 有非常 strong 的 **Data Scientist**

Phase 1 — Internal Startup Project (2024 年初到年中)

刚开始的时候,Generative SERP 有点像一个创业项目。

- 不是 **official project** —— 我们 official 已经有很多事情在做,有非常 clear 的 OKR
- 要做 Generative SERP 意味着要去 **squeeze resource**
- 但我觉得是值得花精力去做的

Pitch CVP 的过程

我跟 US Leads work together,准备了一个非常 high-level brief 的 proposal。

比较好的是我们的 **CVP** 也有类似的想法,只是非常朦胧没有成型。看了 proposal 之后: - 觉得更加具象化 - 也给了很多建议 - 同时非常清晰地告诉我们 —— 现在没有 **resource**

CVP 的 boundary: - 想做 innovation 必须 squeeze - 但在不影响现在 **KR** 的基础之上,他是 support 的

我们的回应: - 确保最重要的 KR 能够实现 - 有些 area 对中长期影响没那么大,可以尽快 wrap up,慢慢 save 一些 resource 去做 exploration

故事的开端就是一个公司内部 **Startup Project**。我们没有 resource,需要 form 一个非常 agile 的 team 去 implement。我这边 Team 负责 Content Quality,美国 Partner Team 负责 UI 上的设计。

Phase 1 Roadmap — 找 Driving Scenario

1. 找到 Driving Scenario: - 在哪些 segment 里、哪些场景下,现在的 SERP 是完全没有办法或只能 partially 满足? - 找到这样的 **Killing Scenario** - 觉得用 squeeze 这些 resource 应该可以做到

2. 分析用户满意度: - 通过各种 hard 分析和对用户 UI 的满意程度 - 试着 summarize 一些场景,比如: - **Travel 场景 - Health 场景 - How-To 场景** 等

目标 Vision —— 博物馆般的体验:

我们希望真色搏(Generative SERP)能够变成一个像博物馆一样的体验,对用户来说它是一个**最优的文档**: - 有文字 - 有 Answer Card - 用户可以使用内容 - 通过跟 Answer Card 的链接进行交互 - 也可以帮用户做进一步探索 - 或者直接点击了一些链接,可以 take action

Step-by-Step 朝 Optimal Document Magazine Experience 走。

Phase 1 Execution

- 用 ~1 个月时间在 2-3 个 parallel scenario 上做出 demo
- 不能说完全自动化 —— 通过各种 manual 和半自动化形式,对 segment query 产生比较 rich 的结果
- 刚开始每产生一个 query 花的时间比较长
- **Strategy: Query First** —— 先证明可行,再反过来精简 pipeline 和 model 的使用,使其达到很低的 latency

Phase 1 SVP Review

正式跟 SVP 进行 review: - 看到 visualize 的场景之后,觉得非常满意 —— 就是他想要的形式 - 给了一些建议: 应该怎么去衡量 **quality** - 这些是探索中暂时没有放进去的(觉得是后面的问题) - 但 **SVP 强调:如果没有 quality guardrail,后面 launch 会遇到很多问题** - 建议先在 parallel 开始想这一块,因为它是周期性比较长的事情

这个 pitch review 比较成功,我们也得到了一些 concrete feedback 可以去 refine 我们的 plan。

三阶段 Roadmap

Phase 1: Implement 完全 offline,不惜成本构造 optimal document - Online serve 时可以用 offline 生成的 cache,Online 用 key-value 做 similar query retrieval - **第一步 ship 的 Roadmap**

Phase 2: 不能局限于完全 offline,因为不可能 cache 很多 - Offline 作为其中一个 approach,同时需要 **build online story**(包括 evaluation story) - 多个系统 serve generative 的过程

Phase 3: 除了 template-based UI approach,希望能够更加 **innovate move to Generative Experience**

大概是从 **2023 年下半年到 2025 年上半年**做的一些事情。

从 Incubation 到 Official Investment 的转变

第一阶段 hero scenario showcase 之后: - Leadership 已经有了更坚定的信心,愿意让我们挪更多 resource 过来 - 当第一个 offline cache 方式的 MVP launch 后,意识到我们真的要 **seriously 投入 online story**

Resource 大幅调整: - 在那个时间点开始 prioritize 所有项目 - 有了 Leadership support 后,propose 了一些 de-prioritize 的 project,或把项目转移到其他团队 - Free 出来的 resource 全力以赴在 Generative SERP - 后期我把 2-3 个项目都彻底交了出去 - 整个团队腾出了 ~2/3 的 resource 放在 **Generative SERP** 上 - 美国团队那边也类似

结果: - 真的把一个项目从早期阶段带入正式轨道 - 把团队整个 investment 方向和 structure 都因此做了调整 - 后来这个团队成为了整个 Bing 核心的 GenAI 团队 - 也是 Copilot Search 的早期形态产品 —— 为后来更多和 Copilot 集成及扩展的事情打下良好基础

Part 2: Hallucination Evaluation —— 团队最 Critical 的挑战

整个 Generative Search 建模过程中,有很多维度:hallucination、relevance、是否重复、是否详尽、是否对用户有用等。

最有挑战性的就是 Hallucination。

- 大模型刚出来时,hallucination 是被诟病较多、挑战很大的问题
- 想在 Bing SERP 这么一个高流量入口上塑造直观的、端到端的 **visual component 体验** —— 控制好 hallucination 就变得极其重要

内部两大派的对立

包括 PM、TPM、US Partner、自己的团队、公司的 Moderation Team —— 大家分成两大派:

- 1. 保守派:** - 必须有非常严格的 hallucination 定义 - 将相关权重降到极低
- 2. Velocity 派:** - 在目前的大环境下不能过度追求极致 quality - 应该先让用户体验产品形态,收集一手反馈 - 通过迭代慢慢达到理想状态 - 只要 hallucination 表现跟竞品类似或稍低,用户就不至于产生巨大抱怨

我作为区域 Lead 必须 Make The Call

- 最终 Content Quality 的压力还是会来到我这里
- 我觉得这不应该是一个非黑即白的 01 问题
- 不是只能在保守和激进之间二选一
- 希望找到一种方法 —— 既能保证 **Safety**,又能兼顾 **Velocity**

Step 1: Diverse Metrics —— 多样化指标体系

在执行过程中提出 diverse metrics:

测量问题的上下限: - (a) 在极度严格的定义下,查看产品形态和表现 - (b) 在相对宽松 (loose) 的定义下,观察 hallucination rate 的变化

内外部对照: - (a) 观察我们自己内部迭代的不同版本 (treatments) - (b) 重点参考外部竞品目前能达到的程度

Pointwise + Pairwise: - 定义了很多 pointwise feature 测量 hallucination - 涵盖严格和宽松的不同标准 - 包括我们自己和 partner 的数据 - 进一步构建 **pairwise 指标** —— 在迭代过程中通过 SBS 方式具备评估不同版本之间直接比较的能力 - 对 launch attention 决策非常有帮助

Step 2: User-Aligned North Star

在这个过程中,我们不应该在刚开始的时候太过于考虑所谓 hallucination threshold 到底设计在哪。
实际上应该去看在不同的 **hallucination rate** 下,用户能看到的生成内容到底是什么样子的。

确认两件事: 1. 内容是否和用户预期及接触范围是 **aligned** 的 2. 在不同定义下 (比如 5% hallucination) 用户真实阅读并利用这些内容时,会带来什么具体情况

和用户预期 **align** 应该是我们的 **North Star**。

我们要从用户角度去解决问题: 如果我们有一些 hallucination,它是否会真正影响到用户,以及这是否是用户最 **care** 的部分。

这应该是最终的 north star,而不是单一地从两个极端中做选择。

Step 3: 第三种想法 —— Tier 区分 (打破僵局的关键)

做了一些实验数据对比和 label 之后,通过 SBS tool 发现 —— 可以对 **hallucination** 进行分层。

提出第三种想法 —— 根据内容的重要程度设置不同的 **tier**。

两类内容:

1. **直接回答类 (Top Tier / High Priority):**
 - 直接回答用户的 answer
 - 或者对信息至关重要的描述
 - 这类内容无论如何都不能出现任何问题 —— 否则会误导用户
2. **支撑性信息 (Supporting Message):**
 - 有些与得到答案直接相关,非常关键
 - 有些只是发散性思考,引导用户进一步扩展,属于更深入阅读

Tier 区分的逻辑:

用户在生成式搜索上,第一步最核心的需求是找到最关心的信息。

如果把所有 metric 都搞得特别严格 → **trigger rate** 特别低 → 标榜了 **fancy** 产品但 **exposed user volume** 实在太小 → 最后变成 **no one uses**。

分层评估框架

1. **事实抽取:** Response 出来后,先抽取其中的 fact
2. **权重评估:** 评估每个 fact 对用户 **main intent** 的贡献程度,以及对用户认知和后续 exploration 的影响
3. **分层设定容忍度:** - **Top Tier (High Priority):** 最核心 hallucination 问题,tolerance 非常低,希望将 defect 降到 **1% 以内** - **Supporting Statement:** 即使有一些 hallucination,对用户核心意图的理解影响不会太大。可以放宽条件,比如**阈值限定在 5% 以内** —— 可接受范围

后续 Action

基于这个标准,让 MLE 和 Data Science team 做了一些实验,根据实验结果重新定义 triggering threshold:

1. **同步数据:** 把目前 number share 给各个 leads,听取建议
2. **Bug Bash:** 设置 bug bash,邀请用户测试并提供 feedback
3. **定向抽样:** 专门 sample 一些 supporting evidence 中可能存在 hallucination 的 case 给大家看,评估这是否会带来严重实际问题

5-6 次 Leadership Review 收敛

把这些信息汇总之后,前后给我们的 Leadership Team 做了 5-6 次 review。

他们非常关心 quality —— 这直接关系到能不能正式 launch。

当有了更多 backtesting 和 iteration 后,再去 share proposal 和最新数据时:

- **Severe Tier 1 的 Hallucination Rate 降到 0.5% 以下**
- **Tier 2 的 Hallucination Rate 大概 4-5% 之间**

他们看过这些量级后觉得比较 reasonable。同时也需要结合 (align with) 他们的体感,确认这些内容对用户的帮助程度,以及 hallucination 带来的结果会有什么潜在问题。

Leadership 的几位成员都觉得这是一个不错的选项: 1. 解决了要求极其严格、tolerance 极低的 case 2. 针对 Tier 2 内容,保持了更高一点的 tolerance

Coverage 量依然非常 decent,可以帮我们收集用户反馈的很多一手信息。

Remote Team 的额外挑战 —— 不在 Room 里的会议

很大的挑战: - 很多 senior leaders 在美国 - 随着 Generative Search 到 2024 年中进入 MVP 阶段, hallucination evaluation 变得至关重要 - 由于之前的争论,launch 计划一拖再拖 - Leaders 也希望尽快解决 —— 每周开 1-2 次关于 state 的会议 - 但这些会议时间通常对中国团队很不友好,小伙伴们很难参加

这其实是 remote team 一个普遍存在的问题: 有很多 meeting 我们无法身处其中 (in the room),也就没办法直接施加影响力。

针对这种情况,我采取以下行动:

1. **方案对齐与代理:** - 将关于 evaluation 的想法、idea 和目前背景 - Share 给在美国和我一起 align 做 Generative Search 的 manager - 希望他能代替我在会上进行解释并争取支持 (get support)
2. **利益相关者沟通:** - 同时与相关 PM 进行沟通,让他也 buy-in 这个事情 - 从而能在会上共同 promote 一个更加折中的方案

如何去 influence 那些自己不在场的会议,确实是非常有挑战的事情。后来,我们在 align metrics 并完成几次 iteration 之后,项目基本进入良好状态 (in good shape)。最终得到了 AI Moderation Team 的 sign-off。有了这个许可,我们就可以正式将其 launch 到 production 环境了。

Part 3: Latency — 从 Offline 到 Real-Time 的 Online Story

第一阶段以 Query 为主,所以 latency 没关系。

实际上后面发现: - 要生成一个真色搏(GenSERP),整个 process run 起来,一个 query 要 run 5 分钟 - 非常长 —— 基本上要靠离线处理 - Online 根本不可能满足用户需求 - 如果要 move online,必须做一些事情

早期的 Bet —— Reverse Optimization

虽然 leadership 说 quality 为主,但我觉得后面势必会遇到的一个情况就是 latency。我们还要反向去做 optimization。

所以在项目有了初步 pipeline 搭建起来之后,就开始 kick off 了一些 efforts:

Bet 1: Small Language Model Training

走到后面,我们真的要 real-time streaming 的时候,必须有一个更加轻量的模型,能够保质保量地做这个事情,而不是说寄希望于这种特别超大量的模型通过 API 的方式去做。

Bet 2: Generation Framework — Serial → Parallel

原来的 **Serial Generation**: - 一个 document 会有多个 section - 串行生成模式 - 优点: 每一步可以参考前一步生成的内容,尽可能避免 duplication,生成的内容是完备的 - 缺点: length 也会很长

新的 **Parallel Section Generation Framework**: - Online 时更切实的做法 —— **Parallel Section Generation** - 每个 section 的生成 —— 用更轻量的 model 去支持 - 这样才能实现一个真正的 real-time experience

Early Incubation

有了 idea 之后,在比较早期就让 team 开始 incubate。因为特别是 model 这一段,不是一个很容易的事情:

- 需要把 training premise 搭建起来
- 收集 training data、iterate training data
- 包括新产品的 design 也一直在发生变化 —— 生成的样式和风格也会变化

小模型训练策略

我们可以用某一个中间的版本作为参照,先把 end-to-end 搭建起来,然后能够去证明 ——

比如基于大模型的迭代版本的某一个版本作为 target goal。

然后看小模型应该怎么训练,应该怎么学习数据,然后能够得到 comparable 甚至 beat 这个大模型的效果。

Beat 的可能性: - 大模型我们没有用任何 domain-specific 和 task-specific 的数据去做 fine-tune - 但小模型虽然小,我们可以收集大量的真实数据,让它做 domain learning、alignment - 觉得它是很有希望可以去超越的

飞往 Seattle —— 趁热打铁

第一个版本基于 **Offline Cache ship** 之后: - 很快面临 challenge —— **offline** 生成的量实在太有限 - Leadership 也会 challenge: “我怎么能够 on-demand 生成一个 GenSERP?” - 没有一个让他能去 try 的 pipeline - 经常做法是:让他把 query 给我们,offline 手动生成一个答案 - 这种无论从我们 debug purpose、leadership review,还是长期 skill —— 都是有问题的

这个时间点的状态: - 我这边的 prototype model training 和 parallel section generation 框架已经搭得差不多了
- 所见即所得 —— 把它 visualize 展示出来

我的 Action: - 因为这个问题的的重要性,从北京直接飞到西雅图,跟我们的 Leader 当面做一些 demo - 做了两个 demo: - 一个在 Laptop 上的 demo - 另一个在 手机端的 demo - 给用户、给 Leader 看

Review 成功 — 给小模型起名 “Glow”

- Review 非常成功 —— 这就是他们一直想要的效果,real-time
- 同时给系列小模型起了一个名字叫 Glow

Glow 的 Vision:

我们希望小模型的训练 —— 第一步当然是希望能够把这个东西用在我们整个 serve 这个项目里面,但它应该是一个 framework,应该是一个最终是一个 black box,能够去 serve 我们整个 org 更多的对小模型的需求。

这时候觉得无论是从短期对 GenSERP 项目,还是长期对整个 business —— 都是非常好的 efforts。

从 Incubation 到 Official Investment 转换

在这种情况下,我们也慢慢地从 incubation 状态(少量 resource 投入),变到了一个 official investment。

我的 proposal 也从最初的一个个人提案,演变成了我们整个大的 V-Team 甚至整个 Org 的一个 visionary direction。

Result

后来 Parallel Section Generation + 小模型方案 支持了 Real-Time Generation 的 launch:

- Glow Training Framework 被各个团队广泛采用,甚至一些 Bing 团队之外的 Partner Team
- 很多 Team 做的小模型可能是传统任务,但我们这个才是真正的 generative 任务
- 我们需要兼顾 Bing 这种 high-traffic 场景
- 对 partner team 来说,都是非常好的借鉴和一手资料

具体 impact: - (a) 后续有不下 10 个项目 leverage 了这个 framework - (b) 很多 partner 都是 proactively reach out 找到我们,而不是由我们去 pitch - (c) 他们开始基于我们的框架做调整,并去 contribute 框架本身

这其实达到了一个从项目到 Org Level 的 Impact 转化。

Part 4: Generative UI / Layout —— 第三阶段 Innovation

第三阶段 —— 希望在 experience 上有创新:

- 比较早的阶段,更多是 template-based approach
- 仍然有很多可能性
- 这个时候,可以用大模型帮我们做一个更全局的优化

起初的 Strategy: - 主要把 Generative 的东西作为一个 enhancement - 或者作为 template-based approach 的增强 - 还没有把它发挥到极致

当时的局限: - 模型由于 context length、generation latency 等各方面原因 - 在更复杂的 UI 任务的生成上有比较多的局限性, latency 也比较高

保守策略 → 高质量产品: - 在这种情况下, 更保守的策略会使最终产品质量更高 - 但这种 mindset 以及方向感, 对后面提出更加 scalable 的 GenUI 框架打下了非常好的基础

(GenUI Layer 完整 vision 见下一个故事 Lumina)

故事 10: Jira Search & Agent

Atlassian 当前 Team Structure

我来到 Atlassian, 现在的 team 有三个 charter, 其中非常重要的一块就是 Jira —— 它是我们的 **flagship product**。

这块包含两大块内容: - **Jira Search** - **Jira Agent**

Team Scope — 直接 Reports 与 Beyond

直接 Report 给我的 Team: 大概 20+ 人

Beyond 的 Direct Reports —— 一个 100+ 人的背景: - Jira Agent 不是独立存在的: - 需要 Platform 的 support - 需要 Data Team 的 support - 需要 Jira Team 的 support - 需要非常多 cross-functional team 的 support - 跟我们合作的 team 涉及到 8-9 个 Org

Three-Tier Contributor Structure

Tier 1 — Hands-on Active Contributors: ~50-60 人 - 我自己的 team ~25 人 - 加上 infra team

Tier 2 — On-Demand Contributors: ~40-50 人

Tier 3 — For Awareness: ~40-50 人 - 我们的 leads - 一些 supporting team - 需要给我们一些建议 - 或我们需要他们 review、sign off 一些事情

加在一起, 现在已经是一个 145-150 人的 team。

核心是 hands-on active contribute 的大概 50-60 人(我自己 team 大概 20+ 人), on-demand contribute 的大概 40-50 人, for awareness 的可能也有 40-50 人。

项目 Vision — Search to Agent 的进化

第二周加入时的项目状态:

- 项目只有 **Search Agent**
- 要负责用户的 **knowledge discovery** 以及 **action** 需求
- 因为最终很多 Jira 用户除了获取信息, 还希望能够完成自己的 **task**
- 怎么把信息获取和最终 **take action** 都 **unify** 起来
- 从 basic search 上升到 **agentic** 系统, 帮助用户去完成最终的任务

Search Quality 是用户长期诟病的地方

- Search efficiency 和 quality 一直是用户的诟病点
- 当然很多老用户已经形成了根深蒂固的习惯,有自己一套可以找到东西的方法:
 - 写 JQL
 - 知道怎么做 keyword
 - 怎么做 filter
- 很多老用户已经形成习惯了

Jira Team — Convince 不容易

- Jira Team **base** 在澳洲
- 比较传统的 **software development team**
- 想要 **convince** 他们去 **make change** 不容易

这是为什么 Atlassian AI 团队成立之后,要做 modern search 的时候,没有直接选择 Jira: 1. **Too big to risk** (Jira 体量太大) 2. 用户习惯问题 3. 跟 Jira team 的 alignment 和复杂的 **cross-org collaboration**

Why Jira Has To Be Done — Math Doesn't Add Up Without It

如果不去 transform Jira surface,不可能达到 **MAU goal**。

因为加了其他 surface 加在一起,它可能都没有 Jira 那么高。

第二周被告知 —— Jira 是后面非常重要的 **focused area**。

第一年的 Goal 与 Three Milestones

12-Month Goal: - 把 modern search land - 把 fundamental search quality 提升上去 - 为 **agent** 做好坚实基础
- **AI search MAU** 从 ~3M 涨到 8M

起点状态: - Team 还没有 - 没有一个 **PM**, 一个 **TPM**, 一个 **Principal TL** 已经先期做了一些调研 - **PM** 很早就和 Jira team 有 reach out,跟他们建立了一些联系 - **PM** 和 **TPM** 一起,有一个搭配的产品方向上的 **proposal** - **TL** 比我早 **1** 个月开始,做 survey、了解情况

Milestones (在我加入之前已设好): - **4** 个月 一个 milestone - **8** 个月 一个 milestone - **12** 个月 一个 milestone

Action — Step 1 & 2: Gap Audit + V-Team in Parallel

I ran two things in parallel from week one.

For the gap audit: - 我 built a simple matrix on paper — *what I know, what I half-know, what I have no clue about* - 横跨 4 个 area: - **Jira's legacy stack** - **Confluence's adjacent stack** - The **cross-org PM/TPM map** - **Jira leadership's risk concerns** - Ran **1:1s** with every major **PM**, **TPM**, and **Principal TL** - Built **3 core mentor relationships**: - 一个 **Principal TL** on the Jira stack - 一个 **TL** from Confluence Search 作为 reuse source - 一个 senior **PM** 知道 cross-functional landscape inside out

关键洞察 (改变了 sequencing):

The biggest gap wasn't technical. It was org alignment. The political surface was bigger than the technical surface. That insight changed how I sequenced everything else.

[节奏 tip: “the biggest gap wasn’t technical, it was org alignment” — 慢一点,这是整段第一个洞察 punchline]

In parallel, 解决 resource problem: - 10 engineers — even fully onboarded — weren’t enough - 没有 try to solve it with my own headcount - **Proposed a V-Team**: - 借 shared assets from Confluence Search to bootstrap - Worked with peer managers across multiple orgs to **assign their top engineers to specific accountable components** - Not as a request, **but as joint accountability with shared credit**

It started with about 30 contributors. By the 8-month milestone, the V-Team had grown to 115 people across 8 partner orgs.

And honestly? I didn’t ask for most of those new people. They came in after they saw the early wins shipping.

[节奏 tip: “I didn’t ask for most of those new people” — 慢一点,听起来像在描述一个真实现象,不是在 deliver 金句]

Action — Step 3: Coalition Council + Two-Page Memo

Even with the V-Team, **I couldn’t drive cross-org alignment alone — eight different orgs**, each with their own roadmap and leadership chain.

Built a small inner circle: - 3-4 个人 —— one technical lead, one PM, one TPM - 每个人都有 cross-org credibility - **We called it the Jira Search Coalition Council**

Three Functions: 1. They co-owned the goal 2. They pushed alignment from inside their own orgs 3. They surfaced political signal early —— resource conflicts, roadmap collisions, leadership concerns —— before any of it became a real blocker

By the end of month 1: - Wrote a **two-page execution memo** - 6-8 month plan + team structure + V-Team dependency map - **Deliberately refused to commit to a full-year plan** —— too much uncertainty at that stage

关键 reflection:

One thing worth saying — the Coalition Council didn’t dissolve when Jira Search shipped. The same partnership has unblocked three more cross-org initiatives I’ve led since.

Action — Step 4: Converting Jira Leadership

The hardest part was **converting the most cautious stakeholder — Jira leadership**.

关键洞察 (回头看最自豪的部分):

Their **real worry wasn’t the technology**. It was user habit change on the company’s most important product. A completely legitimate concern.

[节奏 tip: “Their real worry wasn’t the technology. It was user habit change.” — 这是 Director-level 洞察,慢、清楚]

My approach: - I didn’t argue - 给他们 a **phased path with a kill switch at every stage** - Lower-risk surface first - **Backend before UX** - Each stage shipped only after the previous stage’s metrics held up

Rollout 顺序: 1. Quick Search backend first —— 纯 semantic uplift, **zero UX change** 2. **Quick Search UX** —— minor enhancement, not a redesign 3. **Full Page Search** —— backend and UX coupled, with intent-based routing **and a toggle so users could switch back to the legacy experience anytime**

Result:

That's what flipped them. **Their concern went from blocker to active sponsor.** They started **championing the transformation in their own staff meetings.**

Result

Hit all three milestones on schedule —— **4, 8, 12 months.**

8-month mark: - **Quick Search shipped to all customers with the new UX** - Our **CEO announced it at Team — Atlassian's flagship product event** - **AI Search MAU doubled from 3M to 6M**

12-month milestone (this past February): - **Full Page Search shipped** - Today it's at full traffic - **AI Search MAU is now over 8M — beating the original target**

The V-Team grew to 115 people across 8 partner orgs.

Jira Agent — 自然延伸

从去年 10 月份,我们可以同步启动 **Jira Agent** 的 efforts: - 现在 **V-Team 已经达到 140-150 人** - 跨了 **10 个 org** - 没有 search 的坚实基础,我们很难 smooth 扩展到 agent

Jira Agent 的三个 Surface: 1. **Search Smart Answer** 2. **Universal Chat** 3. **Embedded in Jira**

目标: 帮助用户做各种 Jira knowledge discovery、take actions、完成任务

Reusable Framework

Progressive UX migration framework became Atlassian's new playbook for **AI-driven UX change on legacy products.**

故事 11: Lumina — GenUI Layer Across Atlassian Products

Opening

I'd like to give a story about **noticing a strategic gap that no one was owning** — and **deciding to own it without sufficient authority over the teams I needed to influence.**

This is an example from my latest experience at Atlassian, as it illustrates a case of thinking and execution within our AI initiative.

Situation (45s)

Let's set the scene.

About 3-4 months after I joined Atlassian, I started to look at how AI responses were showing up in various products.

I noticed: - Every AI service — **Search, Chat, Jira, and Confluence** — had a response that was **text-heavy - Markdown streamed on some canvas** - This was fine for chatbots in 2023 - **But it wasn't aligned with our aspirations for 2025 or 2026**

The strategic gap I found on the engineering side:

There was **no engineer at a director-equivalent level** whose mission was to make AI look modern across products.

- Although I heard PM side mentioned the old-fashioned UX
- **I don't think anyone was truly taking it on as a mission**

My Choice:

I had a choice: **stay in my lane, or take on this challenge if I had a good proposal.**

I decided with almost no hesitation to take it on.

Task (30s)

I gave a proposal called Lumina.

The idea: - Instead of purely streaming markdown to users - We could enable AI to generate **structured and interactive components blended with text** - Allow users to **click through, manipulate, and even follow up for actions** - Think of it as a **generative UI layer that can be plugged into any AI product**

Vision was deliberately cross-organizational: - I'm familiar with both **chat and search** - Pilot it in **search**, then in **chat**, then **scale to other products**

What Made It Challenging (30s)

Two hard things:

- 1. Not purely a technical science problem; it's an actual product direction problem** - Needed to gather strong support from PMs - Get **design** to sign off on the roadmap, direction, and investment
 - 2. No sufficient authority over these teams - No direct reporting line** over most of them - Had to **convince these teams to truly adopt it**
-

Action — Step 1: Grounding the Proposal + Early Input (45s)

I took action and was deliberate about the sequence — knowing that **the sequence of cross-organizational vision is equally important as the proposal itself.**

First, I grounded the proposal: - Wrote a **one-page proposal based on my prior work in consumer products** - Which included a generative AI experience - Had some **real artifacts — not just a deck or a page**

Then I had 1:1 meetings with 3 major PMs and designers: - With whom I worked closely - To **gain early input** - This was very helpful — these were some of the first big product ideas proposed in Atlassian

Two helpful suggestions received: 1. **Ensure the generative UI quality is decent and fits into Atlassian's design framework** — UI should align instead of having different styles 2. **Should not limit it to specific products** — should think about a broader, higher-level experience innovation

In enterprise, **they want to address product alignment from day one**. This is quite different from my experience at Microsoft.

Action — Step 2: Tiger Team + Prototype + Loom (40s)

Per my experience in consumer products — purely leveraging a written document was not sufficient.

Formed a low-cost “tiger team” of 3 people: - Build a **prototype within 10 days** - Recorded a **Loom video** to walk through the demo - Showcasing exactly how the product would work and look

Loom over live demo was deliberate: - Live demos require everyone in the same meeting at the same time - **Looms get forwarded across the org** - People watch on their own time - **Senior leads share them with their teams without me being in the room** - That **async distribution is what scaled the proposal beyond meetings I could attend**

Action — Step 3: Co-Author + Broad Sharing + Critical Feedback (50s)

At the same time, **I recruited a Principal Engineer to co-author the proposal with me.**

- He has rich experience in UX and the Atlassian product framework
- I felt it was important to have that **technical backing** — to prove the idea was **doable within our existing ecosystem**

Once I had the refined two-page memo and the prototype video ready: - Shared them broadly with senior leads for feedback - General response was positive **because we had something tangible in hand**

Critical feedback from our Head of Product: 1. Hadn't sufficiently addressed synergies with existing UI components 2. Needed a plan to accelerate execution rather than building everything from scratch 3. Needed to ensure that adding rich AI components wouldn't compromise latency across different surfaces

Addressing these real-world concerns took **about 2-3 months of discussion** to reach ultimate alignment and move into full execution mode.

What I'd Do Differently — Early Lessons (40s)

Reflecting on this, there are a few things I would do differently:

First — Seek Partial Alignment - Instead of waiting for a complete final decision on the entire roadmap - I should have **identified which parts were “solid” and pushed for partial sign-off** - This would have **unblocked the MVP** and allowed us to start learning in production earlier - While the more complex discussions continued

Second — Prioritize Low-Dependency Concepts - We started with **Smart Answers in search because my team owned that page** - This **reduced communication costs** and allowed us to prove the concept quickly

Tracking Adoption — 4 Measurable Dimensions (60s)

Critical Insight Evolved:

In the following stage, I evolved to track adoption in a more measurable way to ensure we were truly moving in lockstep toward execution.

Initial Mistake: - At the beginning, I took “**verbal yes**” from adjacent teams (like Chat and Jira) as strong signals for adoption - **However, after 3 months without a single line of code committed, I realized that a verbal agreement is not a real commitment**

The real metric for adoption is multifaceted. 4 measurable dimensions:

- 1. Code Commits** - Best indicator of adoption is **when partner engineering teams contribute to or leverage our components** - If there are no commits, I know I need to **engage more deeply with their TLs and engineering leads**
 - 2. Product Plan Inclusion** - When PM and design leads **include Lumina in their roadmaps** - It marks the **transition from my personal proposal to an official organizational vision**
 - 3. Organic Citation** - Key signal of success: when Lumina is **organically cited in technical reports, writing, or staff meetings for products where I am not even involved** - This demonstrates that the team truly values the solution
 - 4. Pull versus Push** - Success means moving from “**pushing**” the vision to partner teams “**pulling**” it from us - When teams proactively reach out to ask “**when can we onboard**” rather than “**why should we**” — it proves the vision has taken hold - **By the end of this March, 6 product teams had already reached out to us for onboarding**
-

Final Delivery Status (30s)

-  **Lumina has launched in Smart Answers with positive feedback**
 -  **Quickly adopted by Chat**
 -  **Jira product team is currently in A/B testing — expect to reach GA by the end of this quarter**
 -  **6 additional partner teams already in the onboarding pipeline**
-

Reflection — 3 Lessons That Generalize (45s)

Three lessons I carry forward from Lumina — and they generalize beyond this specific project.

First — Track Adoptions with Measurable Dimensions - A verbal “yes” is not a commitment - A prototype is not a finished skill - **By focusing on measurable dimensions** — code commits, plan inclusion, organic citation, push-to-pull — I can scale this framework to other cross-organizational projects

Second — Prototype Velocity is Currency - In the new AI era, the speed and velocity of prototypes serve as **our primary currency** - Showing a working model is **always better than just writing about it** - It is the **most effective way to convince partner teams to adopt our vision**

Third — Sequencing Matters More Than the Proposal Itself - Pilot in **your own org first to build evidence** - Move to adjacent orgs **only after that evidence exists** - Be honest with yourself about **which phase you’re actually in** - **Most failed adoptions happen because someone is in phase one but talks like phase three**

Closing — Prime Video Transfer (20s)

Honestly, **this is exactly the loop I’d want to run on day one at Prime Video.**

The cross-org surface there — **Bedrock, Studios, Live Sports, Rufus, and Alexa+** — is **even bigger than Atlassian’s.**

You don’t navigate that by **pitching harder.**

You navigate it **phase by phase, signal by signal**.

I'd want to spend the **first 30 days listening before proposing** — but the methodology transfers.

故事 12: Knowledge Services — Entity Linking, At-Mention, Collaboration Graph

Big Arrow for Growth — Knowledge Fundamental Service

Knowledge Fundamental 是 Atlassian 的另一个 **Big Arrow for Growth**, 服务于整个 Atlassian 这样一些 knowledgeable service。

Service 1: Collaboration Graph

目的: Model 公司内部人和人之间的合作关系, 基于各种 engagement signal: - 共同编辑文档 - 共同会议 - Report Line 关系 - 等等

为什么 Collaboration Graph 是 fundamental:

在 Atlassian 场景下, 如果你不了解人和人之间的关系, 以及每个人所处的环境, 其他的 **Search** 和 **Recommendation** 的效果可能都 **not useful**。

举例: - 你搜一个 “Deep Dive” —— semantic 能匹配的 document 实在太多了 - 但如果你知道这个人所处的团队、知道他的 **collaborator** - 这种情况下你非常容易能聚合出对他最可能相关和重要的事情文档 - 他可以进一步阅读和 take action

Cross-Product Usage: - Collaboration Graph 是 **across Atlassian** 各个 product 都在用的 product - Search、Chat、Confluence、Jira 都在利用这个 signal

Service 2: User Recommendation / At-Mention Service

容易理解 —— 在很多 app 里面 @ 某个 person 或 group。

做的就是: People Search + People Ranking

集成范围: - 现在在 Atlassian 的产品中都会 **embed** 这个功能 - 当你 add 一个 person 去 take action 或服务他的 awareness - 在用户 typing 时, 我们能推荐合适 candidates - 让他能尽快定位到想要联系的 **team member**

重要性指标:

这个 service 是 **embedded** 在所有 **Atlassian Service** 里面, 是整个 Atlassian **One of the Top 3 High Traffic** 的 **Service**。

Service 3: Entity Linking — 更 Broad 的 Implicit Need

和 At-Mention 的区别: - At-Mention 是很强的 **signal** —— 基本上就是 @ 一个 person、@ 一个 group - **Entity Linking** 是更隐式的、**implicit** 的对 **entity** 进行 **search** 的需求 - 现在和 At-Mention Service share measure by hand - 但它是一个更 **broad** 的要求 —— entity 类型上, 且因为没有 @ 这样的先导 input, 需要更需要利用上下文去决定

Enterprise vs Consumer 的根本不同

无论是 At-Mention、User Recommendation,还是 Entity Linking Service,都比传统 **consumer space** 更有挑战:

1. **准确性要求高** - 大家对它的准确性要求比较高 - 直接影响到后续工作
 2. **需要更强的 Personalization Signal + Dynamic Ranking** - 同样去搜同一个人名 - 但根据你的 collaborator 信息、合作的项目信息 - 你搜的 **Michael** 和另外一个同事搜的 **Michael**,可能就是完全不同的人 - 需要根据当前上下文 + 用户合作关系 + **Teamwork Graph** 上各种 **relationship** 来 rank potential candidates
 3. **没有 Popularity Signal 可以 Leverage** - 原来 consumer space —— 很多 entity 因为有大量 click, popularity signal 非常重要 - 但 **Enterprise** 里没有那么多 **popular entity** (除了特别 high level 的 leader) - **每个人搜的都比较 tail** —— 因为和你合作、和你相关的可能主要在一个小 group - Click / popularity signal 对于每个人来说可能都不适用 - 更重要的是每个人的 **engagement**、**collaborator** 这些 signal
-

Entity Linking Turnaround — 从 Lowlight 到 Highlight

起点状态: - 在我来的时候,Entity Linking 已经 build 了大概 **4-5 个月** - 它是一个非常 **fundamental** 的事情 —— 不理解用户场景的 entity,就不能 bring in 更多相关 context - 经常会把同名但实际上是不同 entity 的信息杂乱在一起,使用户造成非常难的 hallucination problem

我接触 team 时的 feedback: - 大家对它的反馈不是很好 - 非常难的 hallucination problem

Phase 1 (前 1-2 个月): Listening & Fair Judgment

我 also want to be fair to the team。有时候不同的 **PM**、**TPM**,包括 **high-level leads**,可能他们看的也是一个维度或是某一件事情。我也希望自己能够有个 fair 的 judgment。

Action: - 听到这些 feedback 时,只是先把它记录下来 - 希望在后面的 1-2 个月里: - 一方面了解他们的工作 - 另一方面了解对外的 **partnership** 关系 - 获取内部和外部更全面的 **feedback** - 而不是说快速地因为一些零散的 **feedback** 就 **make any change**

Diagnosis (经历 1-2 个月之后)

听了他们的 page,听了一些 sharing session:

Team Talents 和 Morale 的状态: - ☒ 整个 team 的 talents 还有大家的 morale 是在那里的 - ☒ 整个团队的合作关系还是很不错的 - ☒ 想把事情做好的 **motivation** 是很强的

主要问题在 Manager 和 TL: - 没有类似相关的经验 - 处于一种边做边摸索的状态 - 理念也没有和最新的 **AI 技术** 吻合,处于相对滞后的状态

这导致的连锁反应: - Team Members 非常焦虑,有一定的焦虑因素 - 他们没有看到 **Manager** 和 **TL** 有更 **promising** 的方向 - 他们做的事情就会偏 **engineering** 一些

当然,刚开始的阶段先把 service setup 起来,先通过各种 privacy review —— 我觉得 make sense,你至少有一个地基。

但在这时候,如果说 team 仍然长期停留在这种 **data rule-based approach**,那肯定就是有问题。

Phase 2 — 我做的几件事情

第 1 件事 — 理顺 technical 所处的 stage: - Reach out 了很多上游下游的合作伙伴 - 一些核心的例子(特别是有 negative feedback 的) - 了解他们的 **viewpoint**

第 2 件事 — 给 Manager 和 TL 机会去 Development: - 因为我有 Microsoft Entity Linking 经验,非常清楚在新 AI 环境中,这个系统应该往哪个方向 move - 给了他们一些 input,分享了一些相关 material - 希望他们能从这里快速 learn,然后和 team 一起 move

第 3 件事 — Hiring: - Team family 虽然不错,但确实缺少做 Modeling 的 talents,特别是 Knowledge Area 的 modeling - 希望能 target 招到至少 3 年以上经验的 teammates,来带领团队的结构和思维 - 不然完全靠大家一起摸索,可能会走不必要的弯路 - 我自己也不可能时时刻刻 baby sitting - 花了 1.5-2 个月,招来了 2 个 strong talents - 也 leverage 了之前的 social connection

Phase 3 — Manager Out 决策 (Make A Call at Month 2.5)

在这个过程中,我一直在观察 Manager 和 TL 的状态。虽然他们想做一些变化,但在 mindset 和执行上,远远落后于我的 expectation。这已经影响到整个 team 的 productivity 和 development。

我觉得不能再这样下去,应该尽快 take action。

Month 2.5: 开始跟 HRBP 探讨 Coaching Program 和 Performance Management Program 的运行方式和周期。

Month 3 初: - 因为情况没有好转 - 但好消息是招到了 2 个 senior 的、有经验的 deliver people 已经 onboard 了 - 觉得这是一个 good timing 去 kick off progress

Run PM Program: - 开始 follow HRBP 的 guidance - 和他们一起 run 为期一个月的 Performance Management Program - 大概在第 2 周或第 3 周的时候 —— just coincidence,Manager 和 TL 都决定 quit,选择离开

物色新 Leadership

- 时间比较着急
- 有一段时间,TL 是从其他项目内部 transfer 过来的
- 但 Manager 需要从外面招
- 所以花了一段时间直接 take 了 Manager Role,和 TL 一起 work together

后续执行 Smooth — From 50% → 90% Precision in 6 Months

Fix Management 和 Direction 的问题之后: - 把方向 setup 好之后,moving 就非常 smooth - 整个 team 瞄准了正确的方向 —— 从 modeling、pipeline、migration、latency 一点一点去做

6 Months 内的 Transformation:

在 6 个月的时间里,我们把最 top 的 NZ Type 的 Linking Precision —— 从原来只有 40-50% —— 提升到了 85-90%+。

Leadership 反应: - 合作的 leadership 和 partnership 都表示非常 surprised —— 或者说 impressed by Entity Linking team 这半年的执行效率 - 他们根本无法想象在接下来的 6 个月,Entity Linking Team 能 deliver 什么可用的东西

Critical Phase — 与下游 Chat Team 的 Integration 危机

最大的难点: 和下游 Chat Team 的 Integration

为什么重要: 在 Chat 中理解 Entity Link 非常重要 —— 因为我们的 Entity Link 不 ready,之前他们直接用大模型想办法去抽 entity,做 Entity Linking 的 simulation

他们的做法的问题: - 用了多次大模型 - 做了很多 hack 的 post-processing - 效果不长,也不是特别好 —— 可能比我们最早的 40-50% 的 quality 稍微好一点 - 但远远没有达到他们的预期

Convince Chat Team 的过程: - 后来 convince 了他们 - 证明我们的东西比他们原来 hack 的 solution 都要好 - Latency 也很低 - 他们也 buy-in 了 - 一起 work together,把东西 integrate 到 Chat 里面

最 Challenging 的阶段 — Chat Team 想 Take Over

他们觉得 Entity Linking 完全不行,想 take over。

我当时觉得是一个非常 risky 的阶段。

Misalignment 的发现:

跟他们的 Lead 聊,了解到他们的一些误区: - 他们把很多相似的问题,或者但凡涉及到 entity 的问题,都归纳为 Entity-Linked Problem - 我觉得这过于宽泛 - 是一个不合理的认知和错误的预期

我的应对: - 试图用合理的方式去 align 对 Entity Linking 的预期 - 他们提出的其他问题,我觉得应该有其他方法解决 —— 我们也找到了一些 proposal 和 POC - 我们 team 在 align 之后,希望他们能更专注地解决 Entity Link 和 Resolution 的问题 —— 因为这个问题本身就很难 - 如果混在其他问题中一起解决,难度会更复杂,也难控制 quality

Knowledge Workshop (去年 11 月) — Re-Build Trust

为了和 major concern 的 Partner Team —— Chat Team —— 建立联系,我主动 organize 了一个 Workshop。

Workshop 目的: Mutual sharing and alignment

当时状态: - 整个 Entity Linking 的 org 问题才刚刚开始解决 - 但觉得我们必须得有一个 visionary 的东西出来 —— 一个 proposal,一个 demo

Workshop 内容: - 跟大家好好讲了目前的阶段,以及我们怎么 frame 这个问题 - 讨论了下个阶段要做什么 - 包括非常重要的环节 —— Live Demo

Live Demo 的 Trade-off: - Latency 会稍微长一点(用了更 heavy 一点的 model,还没有时间优化延迟) - 但在 initial prototype 里,大家已经能看到 —— 对于原来解决不了的问题: - 引入更多 context 并升级 model 之后 - 都有一个非常明显的提升

Partner Team VP 的态度 Flip

这给 partner team 的 VP 留下了非常深刻的 impression。

他觉得从那一刻开始,这个 team 终于迎来转机了。

从那个点之后,他也没有再继续 **complain**,而是给了我们一些时间,去把 **proposed idea** 做出来。

现在 — 强烈 **Trust** 和 **Expansion**

现在经过这半年的时间 and 更多的讨论,他们对我们已经建立了一个非常强烈的 **trust**。甚至希望把这种 **collaboration** 关系慢慢扩展到 **Entity Linking** 之外的空间。

Entity Linking 的 **Impact**

Entity Linking 的 **impact** 非常大 —— 从 **Search、Chat、Confluence** 到 **Jira**,各个地方但凡涉及到这种需要理解 **context** 来帮助用户提供 **insights** 或 **take action** 的场景 —— 它都需要。

所以它是一个非常 **fundamental** 的事情,影响半径也非常大。

Reusable Methods

- **Be fair to the team before you act** —— Phase 1 的 1-2 个月 **listening** 是必要投资
 - **Diagnose Manager / TL gap separately from team capability** —— 团队 **talent** 在,问题在 **leadership**
 - **Hire ahead of difficult conversations** —— 招到 **senior talent** 才能给 **underperformer** kick off PM,这样无论结果如何 **team** 都不会断
 - **Workshop + Live Demo as Trust Recovery** —— when partner wants to take over, organize an alignment forum with concrete artifacts
-

故事 13: **AI School China for AI Talent Cultivation**

项目缘起 (2017)

AI School 是从 **2017** 年在 **Microsoft** 发起的关于内部 **AI** 人才培养的项目。

最早的限制: - 在美国总部提出 - 要求 **onsite** 参加,所以当时在中国无法参与

当时的 **Industry Context**: - 那时 **AI** 和 **Deep Learning** 刚刚兴起 - 我们看到了它的能力,但远未像现在这样全面开花 - 微软较早意识到 **AI** 人才培养对数字化转型的重要性

中国 **AI School** 的发起: - 因为在中国无法参加美国的 **AI School** - 时任全球 **EVP** 的 **Harry Shum** 建议中国工程院和研究院院长共同推进

我作为第一届学员

我很荣幸成为 **AI School** 的第一届学员。我的直属老板负责执行、推进和整合资源。

第一年初衷: - 第一年是非常初步的探索 - 我作为第一届学员参与其中 - 初衷是不仅学习,更重要的是观察整个流程的问题,并帮助推进 **AI School** 的发展

角色转变 (Year 2)

有了第一年的亲身经历后,从第二年我转变了角色: - 成为 **AI School** 中国的 **Cooperation Team** 成员 - 根据第一年体验,向老板们提出了新的可能形态 - 包括 教师团队构建 和 课程体系安排

第二年的一个重要 **Note**:

完全是 **side project** —— 利用个人时间支持这个项目。

虽然老板在资金上给予了大力支持,但个人精力投入和沟通协调需要自己衡量。

Year 1 的 Limitation

第一年形式比较初步: - 邀请 AI 领域有经验的团队进行讲解分享 - 非常有帮助 - 但在兼顾大家需求方面有局限

Year 2 的 Improvement

因此,在第二年我提议: - 需要更系统的课程 - 邀请资深研究员开发更进阶的课程 - 实现从入门到进阶的 **step by step** - 当时 AI 刚起步,很多人没接触过,这种普及性的课程能带来更大帮助

Year 2 的执行: - 没有发散到太多方向 - 在老板支持下,请到了研究院一位在 AI 教学方面非常有经验的研究员 - 他带领团队开发了一门针对初学者的课程 - 我负责 **Operation**,与他们一起规划流程

Year 2 的 Painful Iteration — Pacing Problem

遇到的问题: - 原计划是先完成课程再实践 - 但很多学员不是全职参加 - 战线拉长后,他们觉得只学理论没用 - 出现焦虑甚至退课

Reflection 与 Adjust: - 我们反思节奏设计有问题 - 通过 **feedback** 发现大家希望边学边做 - 与 **research lead** 讨论,如何快速调整课程,将实践性内容穿插到理论课程中 - 让大家能边学理论边实践,更早上手

后期成果 + Foundation

后期第一门课程进展顺利: - **Operation** 过程是摸索 - 与 **Finance Team** 讨论毕业形式和考察量化 - 为后面几年的 **AI School** 运作打下了坚实基础

Year 2 数据: - 有 ~100 名学员参加 - 角色更多元 - 获得了一手反馈,为未来项目做好准备

Reflection — Project, Not Event

AI School 是一个完整的 **project**,而非一次性 **event**。需要:

- 足够的热爱
 - 清晰的规划
 - 知道每个阶段该做什么
 - 将协调融入日常项目中
-

Year 3+ (2019 onwards) — 完整体系

有了这些积累之后,从 19 年开始新一期的 **AI School** 时,我提出了一个更完整的体系。

3 个更加 **Diverse** 的形式 —— 服务大家不同需求:

Format 1: Lecture (开放式 Talk)

- 每个月 1-2 次 **Lecture**
- 比较 **on-demand, open to all** 的活动
- 大家没有什么限制 —— 你感兴趣这个 **topic** 就过来听

- 跟 speaker 沟通
- 比较灵活的 talk 形式 —— 可以根据热点灵活调整 speaker
- 让大家 feel free 去了解一些 overview、方向 overview
- 不是为了 in-depth 的事情,更多是让大家知道自己,帮自己补一些盲点
- 让大家知道什么东西在当前是比较 popular、是可用的

Format 2: Seminar (研讨会)

- 在公司内部发掘和你有共同兴趣的小伙伴
- 一起有一个探讨和学习的过程
- 实际上是互帮互助的过程
- 希望每一期能成立一些兴趣小组
- 大家在小组里可以共同找到一些学习资料,共同研读、学习和分享
- 最后的 Deliverable 形式可能是:
 - 一个学习报告
 - 也可以是一些 Proposal
 - 如果有时间,也可以做一个真正的项目

Format 3: Course (系统化课程)

- 第三个是这种系统的 Course
- 组织一个更加系统化的教师团队
- 研发一些有针对性的课程,结合热点、结合大家需求
- 对大家的 commitment 要求比较高
- 你真的是要认认真真持续投入:
 - 比如 8 个月学习时间
 - 每周或每两周一次的课程
 - 最后要 deliver 一个 workable 的 project
- 这个有更强的 commitment

Course 的 Learning Outcome

想想看 —— 如果你真的能够在 commitment 的时间里认真去学习的话:

你最终经过 6-8 个月的时间,可能真的有非常多的小伙伴 ——

从原来对 AI 的理解完全不懂 AI,到开始基于 AI 去做事情,或者说从原来只知道其中的一方面,到对 AI 有一个相对比较更全面的了解。

Cumulative Impact (7 Years)

在过去 7 年时间里:

- 培养了 上万的同事们参加我们的活动
- 参与 Course 拿到毕业证书的,大概有 2000 人

“宽进严出” Philosophy

我们其实是一个宽进严出的这样一个事情。报名参与时,我们尽可能给大家更多的机会。但在毕业的时候,我们还是会有一些门槛。

我们也理解大家在工作中可能有各种 **priority change**,可能不一定能坚持到最后。

但但凡参加在这个过程中就会学到一些知识。

能够坚持到最后的小伙伴,他能够得到 **recognition** —— 也希望能够给大家未来的工作带来一个很好的铺垫。

我的 Role — 最长的 Side Project

我在整个 AI School 项目中,作为 **Operation** 的整体的整合者 —— 我觉得是我在 **Microsoft** 持续时间最长的一个 **project**,整整 8 年的时间。

然后看到 **AI School** 蓬勃发展,培养出来非常多优秀的 **AI School** 优秀人才。

Operation Team 的体量

AI School 的 Operation Team: - 不是一个有 **report line** 关系的 **team** - 是一个 **virtual** 的 **team** - 而且是一个流动的 **team**

长期 Operation Team 包括: - **Teacher Team** - **Leader Group** (给我们提供 **mentoring** 和 **guidance** 的)

长期保持参与的人数: - ~5-60 个人这样的规模 - 阶段性地做好 **operation** 协调时,一般维持在 ~20 个人: - 教师团队 - 负责 **operation** 协调的团队 - **TA** 团队

整个 **AI School Committee** 的 **Operation** —— 维持在 5-60 个人状态(因为可能 **on-demand** 需要 **support** 和 **guidance**)。

Personal Growth Reflection

这其实对我的个人发展也是非常重要的一部分。

因为我既要去 **lead** 一个 **group**,去 **lead** 一个 **global team** 完成业务上的进展,同时也需要 **AI School** 的 **Impact**。

1-Year Promotion to Principal — 创下 STCA Record

正是因为我在 **AI School** 对整个公司层面的贡献 ——

我在上 **Principal** 这个 **band** 的时候,花了 1 年时间。

这其实是在微软中国的第一个 **Case** ——

我在这之前是没有人 1 年可以升的。

两大贡献结合: 1. **Product** 上面有非常 **solid** 的 **delivery** —— 帮我们把 **BERT Model** 和 **pretrain** 推动了 **QnA** 系统的非常重要的 **milestone** 2. 同时把 **AI School** 塑造到这样的规模,产生了帮助 **team** 的很大影响

Reusable Methods

- **As an early adopter, become an organizer** —— Year 1 就观察 process 的问题, Year 2 就升级为 cooperation team
 - **Project, not event** —— AI School 的 8 年坚持是把” side project”运营成” long-term institution”的样本
 - **多 format 服务多种需求** —— Lecture / Seminar / Course 三层 commitment 阶梯
 - **宽进严出** —— Lower the bar to participate, hold the bar to graduate
 - **Talent Cultivation 是 Director-level 的 cross-cutting investment** —— 从公司层面贡献 vs 单一产品 delivery, 两者结合才得到 1-year Principal promotion
-

附录 A: Story-to-Interview Mapping (面试题类型映射)

目的: 把上述 13 个项目按” 面试可用性” 打个 mapping, 在准备 HM Round / Bar Raiser / Hiring Loop 时知道哪个故事用在哪种问题上, 效率最高。

Question 1 — Outside Comfort Area (能力延展)

故事	适用度	关键 anchor
故事 10 — Jira Search & Agent	★★★ Primary	第二周接手 / 60% revenue / 4-8-12 month / 3M → 8M / V-Team 30 → 115 (后期 140-150)/ Coalition Council / leadership skeptic conversion / “biggest gap wasn’t technical, it was org alignment”
故事 4 — Multilingual QnA	★★ Backup	6 个月先行 globalization 战略 / 业界只 1-2 papers 的学术空白 / DGX-2 V-Team / 2 个 CTO Distinguished Award
故事 8 — Undersider	★★ Backup	Edge 跨部门 V-Team / influence vertical/slow-cycle partner / 远程团队的 ally network / quarterly → monthly cycle 牵引
故事 1 — Cross-Region/Remote Work	★ Anecdote	“5 → 40+ team scale”, “Compress and Extend” 哲学

Question 2 — Solving Unknown Problems (未知挑战)

故事	适用度	关键 anchor
故事 6 — Knowledge Card / Interesting Facts	★★★ Primary	2 teams 之前失败 / 16% → 70% (4x) / Document-Based Approach 重构 framing / Diverse Role-Play eval
故事 9 — GenSERP Hallucination Quality	★★★ Primary alt	No industry reference / 两大派对立 / Tier-wise reframe(Tier 1 < 0.5%, Tier 2 ~5%) / 5-6 次

故事	适用度	关键 anchor
		leadership review / 远程团队 ally 代理
故事 4 — Multilingual QnA	★★ Backup	三选项 / 选 Universal 路线 / 2 个月 DGX-2 决策
故事 12 — Entity Linking Turnaround	★★ Backup	40-50% → 85-90% in 6 months / Manager-out 决策 / Workshop 重建 trust / 听了 1-2 个月才动手
故事 3 — First BERT Launch	★ Anecdote	2 天 evaluation / Knowledge Distillation 创新 / 2 周 launch

Question 3 — Driving Adoption of Vision (推动 vision 采纳)

故事	适用度	关键 anchor
故事 11 — Lumina	★★★ Primary	没有 reporting line / Loom prototype 异步推动 / 6 product teams onboard / 4 measurable adoption dimensions (commits, plan inclusion, organic citation, push-to-pull) / “verbal yes is not commitment”
故事 9 — Glow / Parallel Section Generation	★★ Backup	Bing 内部 startup project / Squeeze resource / 飞 Seattle demo / 个人 proposal → org-level vision / 不下 10 个 partner team 主动 reach out
故事 5 — NRT Pipeline	★★ Backup	8 orgs cross-team / 2 个 Key Person / Seattle F2F 趁热打铁 / Platform 唯一 demo 项目 / 18% GPU resource
故事 13 — AI School China	★★ Backup (long-term)	8 年 sustained side-project / Lecture/Seminar/Course 三层 commitment / 上万人参与 / 2000 毕业 / 帮助 1-year 升 Principal
故事 6 — IF Methodology Spread	★★ Backup (alt-angle)	从 graveyard of 2 failures 推动 methodology 跨项目复用 / Diverse Role-Play 成为 reusable framework
故事 7 — ORCA Platform	★ Anecdote	Universal QU/DU platform / “一条龙服务” / Customer Obsession for internal devs

Other Interview 题型 (LP-Specific Probes)

LP / 题型

推荐故事

Hire and Develop the Best

故事 10(V-Team / Coalition Council)/ 故事 12(Manager-out + 招 senior talent)/ 故事 13(8 年 talent cultivation)/ 故事 1(5 → 40+ team scale)/ 故事 2(CodeBERT intern → DeepSeek)

Earn Trust (cross-org)

故事 5(NRT 8 orgs)/ 故事 4(MSRA + Platform 跨组合作)/ 故事 11(没有 reporting line 推 6 product teams)/ 故事 12(Workshop 重建与 Chat team 的 trust)/ 故事 8(Edge 团队 quarterly → monthly cycle)

Customer Obsession

故事 10(user toggle, staged customer rollout)/ 故事 7(ORCA dashboard for internal devs)/ 故事 6(Diverse Role-Play 多 persona)/ 故事 9(user-aligned north star for hallucination)

Frugality

故事 5(18% GPU)/ 故事 4(13% GPU vs 3000 → 400)/ 故事 9(Squeeze internal startup resource)/ 故事 8(10-12 人在 100+ 人项目里发挥 high-leverage)/ 故事 1(Compress and Extend)

Bias for Action

故事 5(Seattle F2F 趁热打铁, Christmas deadline)/ 故事 9(飞 Seattle 给 leadership demo)/ 故事 3(2 天 BERT eval, 2 周 launch)/ 故事 10(week 2 commit 4-month milestone)

Have Backbone; Disagree and Commit

故事 9(third path neither camp had named)/ 故事 6(reject “fix existing pipeline”, propose Document-based)/ 故事 4(disagree with cluster approach, push universal)/ 故事 12(Chat team 想 take over 时坚持 align expectations)

Dive Deep

故事 6(诊断 framing 错误)/ 故事 9(metric-human alignment 严谨 / Tier-wise 拆解)/ 故事 3(Knowledge Distillation 推导)/ 故事 12(听 1-2 个月再动手)

Insist on Highest Standards

故事 9(hallucination ship gate, Tier 1 < 0.5%)/ 故事 4(quality before scale)/ 故事 5(coverage AND freshness)/ 故事 12(40-50% → 85-90% precision)

Invent and Simplify

故事 6(Document-based reframe)/ 故事 9(Tier-wise hallucination + Glow small models)/ 故事 8(multi-level index for long PDF)/ 故事 4(Universal model)

Are Right, A Lot

故事 4(选 Universal 路线 vs 大家选 Cluster)/ 故事 9(选 Tier-wise vs 二元保守/激进)/ 故事 1(信息差中的判断)

Strive to be Earth's Best Employer

故事 13(AI School 培养上万人)/ 故事 12(给 Manager 和 TL 发展机会再 PM)

附录 B: Director-Level 操作系统 (Reusable Methods Across Stories)

跨故事的 Director-level reusable methods —— 这些是面试官应该在多个故事中听到的同一个 Director-level 思维。

On entering new domains: 1. **30-60-90 Gap Audit** (故事 10, 4, 12): Days 1-30 context learning / Days 30-60 quick experiment / Days 60-90 framing memo 2. **Listen 1-2 months before acting on a problem team** (故事 12): Be fair to the team —— diagnose Manager/TL gap separately from team capability before making changes

On org alignment: 3. **Org alignment 和 technical depth equally important** (故事 10): On legacy products, alignment comes first 4. **In ambiguity, design the third option no one named** (故事 6, 9): Reframe before choice 5. **When 2 competent teams have failed, audit shared framing** (故事 6): Biggest unlocks come from reframing 6. **When leading from constrained position, small-circle pre-alignment is mechanism of influence** (故事 9, 8): The room you're not in matters more —— allies execute on your behalf in meetings you can't attend

On partnership architecture: 7. **Build a Coalition Council for cross-org initiatives** (故事 10): 3-4 person inner circle that doesn't dissolve after the project 8. **Lasting partnership is the goal underneath the goal** (故事 1, 10, 12): Share credit when right, own when wrong 9. **Treat partner-team POCs as co-authors, not stakeholders** (故事 5): When N partner teams take dependency, change-mgmt ships with technical design 10. **Find the underdog partner first when method needs adoption** (故事 8): 印度 PDF AdSense Team 比美国 main team 更 hungry —— their adoption earns leverage to convert the bigger partner

On architectural thinking: 11. **Build for systematic capability, not immediate feature** (故事 4, 7, 9 Glow): Reframe project asks into platform investments 12. **Two-axis architectural decisions escape "obvious is unaffordable"** (故事 4, 9): Attack 2 constraints simultaneously (Universal model + NRT inference / Tier-wise hallucination + Glow small models)

On AI-native leadership: 13. **Prototype velocity is the currency of influence** (故事 11, 9): Showing beats telling —— Loom video / fly to demo 14. **Async distribution scales reach** (故事 11): Loom video gets forwarded across orgs in rooms you can't attend

On driving and tracking adoption: 15. **Run the Adoption Loop — phase by phase, signal by signal** (故事 11, 5): Permission → Evidence → Pull → Track. **Permission ≠ adoption. Pilot ≠ scale. Verbal yes ≠ commit.** 16. **Track 4 measurable adoption dimensions, not verbal yes** (故事 11 V2.1): (1) Code commits, (2) Product plan inclusion, (3) Organic citation in rooms you're not in, (4) Push → Pull transition (partner teams reach out first) 17. **Sequence matters more than the proposal itself** (故事 11 V2.1): Pilot in your own org first → adjacent org with evidence → broader. Most failed adoptions happen because someone is in phase one but talks like phase three

On hard people decisions: 18. **Hire ahead of difficult conversations** (故事 12): 招到 senior talent 才能给 underperformer kick off PM Program —— 这样无论结果如何 team 都不会断 19. **Workshop + Live Demo as Trust Recovery** (故事 12): When partner wants to take over, organize an alignment forum with concrete artifacts —— 给 VP 留下 visible turnaround signal

On long-term institutional building: 20. **Project, not event** (故事 13): AI School 8 年坚持是把 “side project” 运营成 “long-term institution” 的样本 21. **Multi-format serves multi-commitment** (故事 13): Lecture (open-to-all) / Seminar (interest groups) / Course (8-month with project) —— “宽进严出” 22.

Cross-cutting investment unlocks Director-level promotion (故事13): 1-year Principal promotion 不是单一 product delivery 来的,是 product delivery + AI School org-level impact 的结合

附录 C: Bridge to Prime Video — Story Transferability

每个故事都有一条 transferable thread 可以连到 Prime Video Search & Assistant 角色。在 HM Round 和 Hiring Loop 上,主动把这条 thread 说出来,让 Ming Ye 听到 transferable evidence —— 而不是让她自己去脑补。

故事	Transferable Thread to Prime Video
故事 10 (Jira Search)	高 stakes legacy flagship 上的 cross-org 转型 → Prime Video Search & Assistant 的 cross-org 难度更大(Bedrock / Studios / Live Sports / Rufus / Alexa+),但 same political-audit-before-technical playbook
故事 11 (Lumina)	“No reporting line 但要 influence 6 个 product team” → PV Search 也是新 surface,需要影响 PVPD 内部多个团队和 Bedrock / Rufus / Alexa+ 等外部团队
故事 9 (GenSERP Hallucination)	LLM 时代 quality framework 在 search 上的奠基 → PV Conversational Discovery 的 quality methodology 几乎没有 prior art,可以直接 transfer
故事 4 + 5 (Multilingual + NRT)	Cross-lingual / cross-region scaling + Resource constraints 下的 systematic capability building → PV 全球化 search、不同 device & region 的体验也需要 universal pipeline thinking
故事 6 (Interesting Facts)	“团队失败的 graveyard 里 reframe problem” + Document-based MRC → PV catalog 上的 metadata enrichment / hidden gem surfacing 直接可以 leverage
故事 8 (Undersider)	Bing × Edge 跨部门合作 + Multi-level Index for long doc → PV 内部 Studios / Live Sports / Rufus 也是不同 culture/cycle 的合作伙伴
故事 12 (Entity Linking)	Enterprise scenario 下没有 popularity signal,需要 personalized ranking → PV 也是 user-personalized,collaboration-graph-equivalent 是 watch history + family profile + device history
故事 13 (AI School China)	8 年 talent cultivation → PV 团队 Scale 到 80-100 人需要的 talent strategy + culture building

Document end. V2.1 Update: 13 项目全部详细整理 —— - 故事1-7 来自原文(2026.05 版本) - 故事8 (Undersider): 新增33 段口述,涵盖 Edge 合作、long PDF index、印度团队切入 - 故事9 (GenSERP/Copilot Search): 新增85 段口述,涵盖 Hallucination Tier-wise framework、Glow 小模型、飞 Seattle demo、4 阶段演

进 - 故事 10 (Jira Search): 新增 42 段口述 + EN spoken delivery 版本(节奏tips 标注) - 故事 11 (Lumina): 新增 69 段口述 + 4 measurable adoption dimensions + Prime Video closing - 故事 12 (Entity Linking Turnaround): 新增 21 段口述,涵盖 Manager-out 决策、Chat team 危机、Workshop 重建 trust - 故事 13 (AI School China): 新增 8 段口述,涵盖 8 年演进、Lecture/Seminar/Course 三层、1-year Principal promotion - 附录 A: Story-to-Interview Mapping(扩展到包含 V2.1 新增故事) - 附录 B: Director-Level 操作系统(从 13 条扩展到 22 条 reusable methods) - 附录 C: Bridge to Prime Video(新增,8 个故事的 transferable thread)

文档总长度: 约 65 页 PDF / 大约 11 万字符 / 13 个完整 stories + 3 个 appendices。